Historical Weather Data Analysis & Feature Engineering Using Python

Python • Pandas • NumPy • Matplotlib • Seaborn • EDA • Data Cleaning • Statistical Analysis

Project By: Anurag Kumar Poddar

Data Analyst/Business Analyst

Tools & Technologies: Python, Pandas, NumPy, Matplotlib, Seaborn, Jupyter Notebook

1. Project Introduction

Weather conditions influence many aspects of daily life, including transportation, agriculture, energy consumption, and environmental planning. Analyzing historical weather observations helps uncover recurring patterns, seasonal variations, and relationships between atmospheric variables.

This project explores historical weather data using Python to understand how key meteorological factors, such as temperature, humidity, wind speed, atmospheric pressure, and visibility, vary over time and interact with one another.

The analysis follows a structured data analytics workflow, beginning with dataset inspection and data quality assessment, followed by data cleaning, exploratory data analysis (EDA), feature engineering, statistical analysis, and visualization. Different analytical techniques are used to identify trends, examine correlations, compare seasonal patterns, and investigate relationships between weather conditions.

The overall aim is to transform raw weather observations into meaningful insights and prepare a reliable dataset that can support further analytical work and potential machine learning applications.

2. Project Objectives

The primary objective of this project is to explore historical weather observations, assess data quality, and identify meaningful patterns and relationships among weather variables.

The key objectives are:

- **Data Quality Assessment:** Examine the dataset for missing values, duplicate records, inconsistent column names, and potential anomalies.
- **Data Cleaning and Preprocessing:** Correct identified data quality issues and prepare the dataset for further analysis.
- **Exploratory Data Analysis:** Investigate the distributions, ranges, and variations of important weather variables.
- **Temporal and Seasonal Analysis:** Examine changes in temperature, humidity, wind speed, and precipitation across months and years.
- **Statistical and Correlation Analysis:** Evaluate relationships between weather variables and investigate the proposed hypotheses.
- **Feature Engineering:** Derive useful time-based features to support deeper analysis and future modeling.
- **Data Visualization:** Present key patterns and findings through clear, informative charts and plots.
- **Model-Ready Data Preparation:** Establish a foundation for potential future machine learning applications.

3. Dataset Overview

This project uses a historical weather dataset containing approximately 96,453 hourly observations and 12 columns, covering the period from 2006 to 2016.

The dataset includes numerical and categorical variables describing atmospheric conditions, including recorded temperature, apparent temperature, humidity, wind speed, wind direction, visibility, atmospheric pressure, precipitation type, and textual weather summaries.

The timestamp information enables time-based analysis, while the numerical and categorical features support statistical comparisons and the investigation of relationships between different weather conditions.

Before performing detailed analysis, the dataset is inspected to understand its

structure, identify data quality issues, and determine the preprocessing steps required for reliable results.

Importing Libraries

The first step in this project is to import the Python libraries required for data manipulation, numerical computation, statistical analysis, and data visualization. These libraries provide the tools needed to inspect the dataset, perform data cleaning, explore weather patterns, and identify relationships between meteorological variables.

The following libraries will be used throughout the analysis:

In [4]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
```

Understanding the Imported Libraries

- Pandas: Used for loading, cleaning, transforming, and analysing tabular weather data.
- NumPy: Supports numerical computations and mathematical operations.
- Matplotlib: Used to create charts and visualise weather trends.
- Seaborn: Helps create statistical visualisations, including distributions and correlation heatmaps.

In [5]:
```
df=pd.read_csv('WeatherHistory.csv')
df.sample(5)
```

Out[5]:

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wi Beari (degre
8801	2007-04-01 17:00:00.000 +0200	Partly Cloudy	rain	15.338889	15.338889	0.48	9.6439	3
66901	2013-03-25 13:00:00.000 +0100	Overcast	snow	-0.955556	-6.316667	0.97	19.2878	2
49844	2011-05-15 21:00:00.000 +0200	Overcast	rain	12.200000	12.200000	0.91	21.6384	28
11881	2007-01-18 01:00:00.000 +0100	Breezy and Overcast	rain	9.961111	6.255556	0.70	34.4701	22
42316	2010-11-07 05:00:00.000 +0100	Foggy	rain	5.011111	5.011111	1.00	1.5617	16

Dataset Introduction & Metadata Architecture

The foundation of this analysis is a historical weather dataset containing nearly 10 years of hourly meteorological observations. The dataset includes approximately 96,453 records and 12 important weather-related features.

These features capture different atmospheric conditions such as temperature, apparent temperature, humidity, wind speed, visibility, pressure, and weather summaries. The dataset provides detailed information that helps in understanding long-term weather patterns, seasonal changes, and relationships between different meteorological variables.

This project focuses on performing Exploratory Data Analysis (EDA) on a weather dataset to understand historical weather patterns and relationships between key meteorological variables.

Domain: Meteorology & Predictive Atmospheric Modeling

Objective: Clean, visualize, and statistically audit 10 years of historical weather observations (2006-2016) to build an optimized, model-ready dataset for predicting weather patterns

Core Goals:

1. Identify and fix structural data anomalies (impossible measurements, missing variables, duplicates).
2. Quantify class imbalances within categorical columns.
3. Map thermodynamic relationships (Temperature vs. Humidity).
4. Build a feature-engineering blueprint for future Machine Learning pipelines

Hypotheses to Test:

H_1 (Temperature vs. Humidity): There is a strong negative correlation between Temperature and Relative Humidity.

H_2 (Apparent Temperature vs. Actual Temperature): High wind speeds significantly increase the variance between Actual Temperature and Apparent Temperature (Wind Chill effect).

H_3 (Seasonality): Humidity levels show a statistically significant variation across different seasons

In [6]:
```
#  plotting style
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
```

Feature Understanding

Formatted Date

Represents the timestamp of the weather observation. This feature helps analyze seasonal patterns, yearly trends, monthly temperature variations, and time-based weather changes.

Summary

A short textual description of the weather condition such as Clear, Partly Cloudy, Foggy, or Rainy. It provides a quick overview of the atmospheric situation.

Precip Type

Indicates the type of precipitation occurring during the observation, mainly rain or snow. This feature is useful for understanding weather severity and seasonal precipitation trends.

Temperature (C)

The actual recorded air temperature in Celsius. This is one of the most important weather variables and strongly influences humidity, visibility, pressure, and human comfort.

Apparent Temperature (C)

Also known as the “feels like” temperature. It considers the combined effects of wind speed and humidity on human perception of temperature. During windy or humid conditions, apparent temperature may differ significantly from actual temperature.

Humidity

Measures the amount of moisture present in the air, usually ranging between 0 and 1. High humidity is often associated with cloudy or rainy conditions, while low humidity is linked with dry weather.

Wind Speed (km/h)

Represents how fast the wind is moving. Wind speed plays an important role in temperature perception, storm formation, and visibility conditions.

Wind Bearing (degrees)

Indicates the direction from which the wind is blowing, measured in degrees. This feature can help identify dominant wind directions and seasonal airflow patterns.

Visibility (km)

Measures how far objects can be clearly seen. Fog, rain, snow, or pollution generally reduce visibility. This feature is especially important in transportation and aviation studies.

Cloud Cover (written as “Loud Cover” in your dataset, likely a typo)

“Cloud Cover” is a standard weather feature, while “Loud Cover” has no meaning in meteorology. Also, the column values range between 0 and 1, which matches how cloud cover is measured.** Represents the fraction of the sky covered by clouds. Higher cloud cover is usually associated with lower sunlight exposure and cooler daytime temperatures.

Pressure (millibars)

Atmospheric pressure measured in millibars. Pressure variations are closely linked with changing weather systems. Low pressure often indicates storms or rainfall, while high pressure is commonly associated with clear skies.

Daily Summary

A broader textual description summarizing the weather conditions throughout the day. This feature can be useful for natural language insights and weather categorization.

Data Understanding

In [7]:
```
df.shape
```

Out[7]:
```
(96453, 12)
```

The datset contains 96453 rows and 12 columns.

In [9]:
```
df.head()
```

Out[9]:

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	V
0	2006-04-01 00:00:00.000 +0200	Partly Cloudy	rain	9.472222	7.388889	0.89	14.1197	251.0	
1	2006-04-01 01:00:00.000 +0200	Partly Cloudy	rain	9.355556	7.227778	0.86	14.2646	259.0	
2	2006-04-01 02:00:00.000 +0200	Mostly Cloudy	rain	9.377778	9.377778	0.89	3.9284	204.0	
3	2006-04-01 03:00:00.000 +0200	Partly Cloudy	rain	8.288889	5.944444	0.83	14.1036	269.0	
4	2006-04-01 04:00:00.000 +0200	Mostly Cloudy	rain	8.755556	6.977778	0.83	11.0446	259.0	

In [10…

```
df.tail()
```

Out[10]:

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	W Bea (degr
96448	2016-09-09 19:00:00.000 +0200	Partly Cloudy	rain	26.016667	26.016667	0.43	10.9963	
96449	2016-09-09 20:00:00.000 +0200	Partly Cloudy	rain	24.583333	24.583333	0.48	10.0947	
96450	2016-09-09 21:00:00.000 +0200	Partly Cloudy	rain	22.038889	22.038889	0.56	8.9838	
96451	2016-09-09 22:00:00.000 +0200	Partly Cloudy	rain	21.522222	21.522222	0.60	10.5294	
96452	2016-09-09 23:00:00.000 +0200	Partly Cloudy	rain	20.438889	20.438889	0.61	5.8765	

The dataset contains both numerical and categorical features related to weather conditions such as temperature, humidity, pressure, wind speed, and weather summaries. Initial inspection shows that the dataset is structured properly

Dataset Information

In [11…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 96453 entries, 0 to 96452
Data columns (total 12 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Formatted Date            96453 non-null  object
 1   Summary                   96453 non-null  object
 2   Precip Type               95936 non-null  object
 3   Temperature (C)           96453 non-null  float64
 4   Apparent Temperature (C)  96453 non-null  float64
 5   Humidity                  96453 non-null  float64
 6   Wind Speed (km/h)         96453 non-null  float64
 7   Wind Bearing (degrees)    96453 non-null  float64
 8   Visibility (km)           96453 non-null  float64
 9   Loud Cover                96453 non-null  float64
 10  Pressure (millibars)      96453 non-null  float64
 11  Daily Summary             96453 non-null  object
dtypes: float64(8), object(4)
memory usage: 8.8+ MB
```

Some columns contain missing values that may require preprocessing.

Statistical Summary of Numerical Features

In [12…
```
df.describe()
```

Out[12]:

	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	Visibility (km)
count	96453.000000	96453.000000	96453.000000	96453.000000	96453.000000	96453.000000
mean	11.932678	10.855029	0.734899	10.810640	187.509232	10.347325
std	9.551546	10.696847	0.195473	6.913571	107.383428	4.192123
min	-21.822222	-27.716667	0.000000	0.000000	0.000000	0.000000
25%	4.688889	2.311111	0.600000	5.828200	116.000000	8.339800
50%	12.000000	12.000000	0.780000	9.965900	180.000000	10.046400
75%	18.838889	18.838889	0.890000	14.135800	290.000000	14.812000
max	39.905556	39.344444	1.000000	63.852600	359.000000	16.100000

The numerical weather features have different ranges and variations. Some features like temperature, wind speed, and pressure may contain extreme values or high variability.

Check Missing Values

In [13…
```
df.isnull().sum()
```

Out[13]:
```
Formatted Date               0
Summary                      0
Precip Type                517
Temperature (C)              0
Apparent Temperature (C)     0
Humidity                     0
Wind Speed (km/h)            0
Wind Bearing (degrees)       0
Visibility (km)              0
Loud Cover                   0
Pressure (millibars)         0
Daily Summary                0
dtype: int64
```

Precip Type contain missing values.These missing values will need to be handled during the data cleaning stage.

Check Duplicate Rows

In [14…
```
df.duplicated().sum()
```

Out[14]:
```
np.int64(24)
```

The dataset contains duplicate observations which may affect analysis and should be handled during preprocessing

Final Summary for Data Understanding Section

The initial analysis shows that the dataset is properly organized and contains both numerical and categorical weather features. Some missing values or duplicate may be present, which will be handled during data cleaning before detailed analysis.

Data Cleaning

This section focuses on identifying and fixing issues in the dataset before starting detailed analysis and visualization.

Check Column Names

In [15…
```
df.columns
```

Out[15]:
```
Index(['Formatted Date', 'Summary', 'Precip Type', 'Temperature (C)',
       'Apparent Temperature (C)', 'Humidity', 'Wind Speed (km/h)',
       'Wind Bearing (degrees)', 'Visibility (km)', 'Loud Cover',
       'Pressure (millibars)', 'Daily Summary'],
      dtype='object')
```

Rename Incorrect Columns

As mentioned earlier, “Loud Cover” appears to be a typing error, so the column name is being changed from “Loud Cover” to “Cloud Cover”.

In [16…
```
df.rename(columns={'Loud Cover':'Cloud Cover'}, inplace=True)
```

In [17…
```
df.columns
```

Out[17]:
```
Index(['Formatted Date', 'Summary', 'Precip Type', 'Temperature (C)',
       'Apparent Temperature (C)', 'Humidity', 'Wind Speed (km/h)',
       'Wind Bearing (degrees)', 'Visibility (km)', 'Cloud Cover',
       'Pressure (millibars)', 'Daily Summary'],
      dtype='object')
```

Convert Date Column into Datetime Format

In [18…
```
df['Formatted Date'] = pd.to_datetime(df['Formatted Date'], utc=True)

df['year'] = df['Formatted Date'].dt.year
df['month'] = df['Formatted Date'].dt.month
df['day'] = df['Formatted Date'].dt.day
df['hour'] = df['Formatted Date'].dt.hour
df['local_date'] = df['Formatted Date'].dt.tz_convert('Asia/Kolkata')
```

In [19…
```
df.columns
```

Out[19]:
```
Index(['Formatted Date', 'Summary', 'Precip Type', 'Temperature (C)',
       'Apparent Temperature (C)', 'Humidity', 'Wind Speed (km/h)',
       'Wind Bearing (degrees)', 'Visibility (km)', 'Cloud Cover',
       'Pressure (millibars)', 'Daily Summary', 'year', 'month', 'day',
'hour',
       'local_date'],
      dtype='object')
```

In [20…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 96453 entries, 0 to 96452
Data columns (total 17 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Formatted Date            96453 non-null  datetime64[ns, UTC]
 1   Summary                   96453 non-null  object
 2   Precip Type               95936 non-null  object
 3   Temperature (C)           96453 non-null  float64
 4   Apparent Temperature (C)  96453 non-null  float64
 5   Humidity                  96453 non-null  float64
 6   Wind Speed (km/h)         96453 non-null  float64
 7   Wind Bearing (degrees)    96453 non-null  float64
 8   Visibility (km)           96453 non-null  float64
 9   Cloud Cover               96453 non-null  float64
 10  Pressure (millibars)      96453 non-null  float64
 11  Daily Summary             96453 non-null  object
 12  year                      96453 non-null  int32
 13  month                     96453 non-null  int32
 14  day                       96453 non-null  int32
 15  hour                      96453 non-null  int32
 16  local_date                96453 non-null  datetime64[ns, Asia/Kolkat
a]
dtypes: datetime64[ns, Asia/Kolkata](1), datetime64[ns, UTC](1), float6
4(8), int32(4), object(3)
memory usage: 11.0+ MB
```

The “Formatted Date” column was converted into datetime format to enable proper time-based analysis and trend extraction

Handle Missing Values

In [21…

```
df.sample(10)
```

Out[21]:

	Formatted Date	Summary	Precip Type	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	B (de
5830	2006-03-09 22:00:00+00:00	Foggy	rain	2.266667	-1.994444	0.80	17.4041	
1671	2006-12-17 14:00:00+00:00	Foggy	rain	0.022222	-3.133333	1.00	9.4990	
35189	2010-04-13 03:00:00+00:00	Overcast	rain	10.372222	10.372222	0.80	10.9158	
51592	2011-10-26 15:00:00+00:00	Mostly Cloudy	rain	12.916667	12.916667	0.59	19.8835	
43620	2010-09-28 10:00:00+00:00	Partly Cloudy	rain	19.788889	19.788889	0.54	7.7763	
17396	2007-09-30 18:00:00+00:00	Partly Cloudy	rain	15.055556	15.055556	0.74	3.0590	
65057	2013-07-10 15:00:00+00:00	Partly Cloudy	rain	29.055556	29.250000	0.46	8.2271	
26461	2009-04-14 11:00:00+00:00	Overcast	rain	15.000000	15.000000	0.55	3.2200	
33241	2009-11-22 01:00:00+00:00	Foggy	rain	7.222222	6.161111	0.96	6.4400	
57160	2012-06-15 14:00:00+00:00	Partly Cloudy	rain	27.150000	26.694444	0.34	7.1645	

Handle Missing Values

In [22…

```
df['Precip Type'] = df['Precip Type'].fillna(df['Precip Type'].mode()[0])
```

As observed in the data understanding section, the “Precip Type” column contained some missing values. Since it is a categorical feature with rain as

the most frequently occurring category, the missing values were replaced using the mode of the column.

Check Duplicate Rows

In [23…
```
df.duplicated().sum()
```

Out[23]:
```
np.int64(24)
```

In [24…
```
df.drop_duplicates(inplace=True)
```

In [25…
```
df.duplicated().sum()
```

Out[25]:
```
np.int64(0)
```

Duplicate rows were removed because repeated observations can create redundancy and may affect the accuracy of analysis and visualizations

Final Summary for Data Cleaning

After data cleaning, the dataset became more consistent and ready for exploratory analysis. Missing values were handled, incorrect column names were fixed

Univariate Analysis

In Univariate analysis we anaylsis one column at a time” We focus on:

Distribution,

Central tendency (mean/median)

Spread (std, range)

Skewness

Outliers

Dominant categories (for categorical data)

In [26…
```
df.columns
```

Out[26]:
```
Index(['Formatted Date', 'Summary', 'Precip Type', 'Temperature (C)',
       'Apparent Temperature (C)', 'Humidity', 'Wind Speed (km/h)',
       'Wind Bearing (degrees)', 'Visibility (km)', 'Cloud Cover',
       'Pressure (millibars)', 'Daily Summary', 'year', 'month', 'day',
 'hour',
       'local_date'],
      dtype='object')
```

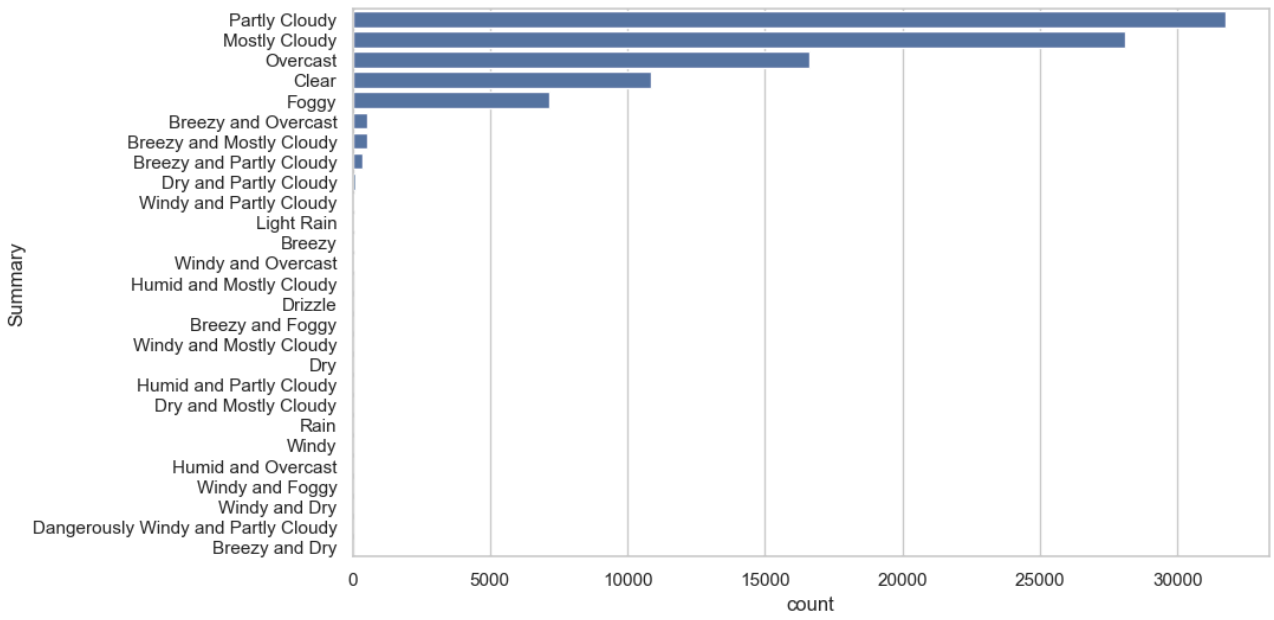
In [27…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 96429 entries, 0 to 96452
Data columns (total 17 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Formatted Date            96429 non-null  datetime64[ns, UTC]
 1   Summary                   96429 non-null  object
 2   Precip Type               96429 non-null  object
 3   Temperature (C)           96429 non-null  float64
 4   Apparent Temperature (C)  96429 non-null  float64
 5   Humidity                  96429 non-null  float64
 6   Wind Speed (km/h)         96429 non-null  float64
 7   Wind Bearing (degrees)    96429 non-null  float64
 8   Visibility (km)           96429 non-null  float64
 9   Cloud Cover               96429 non-null  float64
 10  Pressure (millibars)      96429 non-null  float64
 11  Daily Summary             96429 non-null  object
 12  year                      96429 non-null  int32
 13  month                     96429 non-null  int32
 14  day                       96429 non-null  int32
 15  hour                      96429 non-null  int32
 16  local_date                96429 non-null  datetime64[ns, Asia/Kolkat
a]
dtypes: datetime64[ns, Asia/Kolkata](1), datetime64[ns, UTC](1), float6
4(8), int32(4), object(3)
memory usage: 11.8+ MB
```

Analysis categorical columns

Summary

In [28…
```
sns.countplot(y=df['Summary'], order=df['Summary'].value_counts().index)
plt.show()
```



In [29…
```
df['Summary'].value_counts()
```

Out[29]:
```
Summary
Partly Cloudy                           31726
Mostly Cloudy                           28094
Overcast                                16597
Clear                                   10873
Foggy                                    7148
Breezy and Overcast                       528
Breezy and Mostly Cloudy                  516
Breezy and Partly Cloudy                  386
Dry and Partly Cloudy                      86
Windy and Partly Cloudy                    67
Light Rain                                 63
Breezy                                     54
Windy and Overcast                         45
Humid and Mostly Cloudy                    40
Drizzle                                    39
Breezy and Foggy                           35
Windy and Mostly Cloudy                    35
Dry                                        34
Humid and Partly Cloudy                    17
Dry and Mostly Cloudy                      14
Rain                                       10
Windy                                       8
Humid and Overcast                          7
Windy and Foggy                             4
Windy and Dry                               1
Dangerously Windy and Partly Cloudy         1
Breezy and Dry                              1
Name: count, dtype: int64
```

In [30…
```
print('Unique weather Summaries:',df['Summary'].nunique())
```

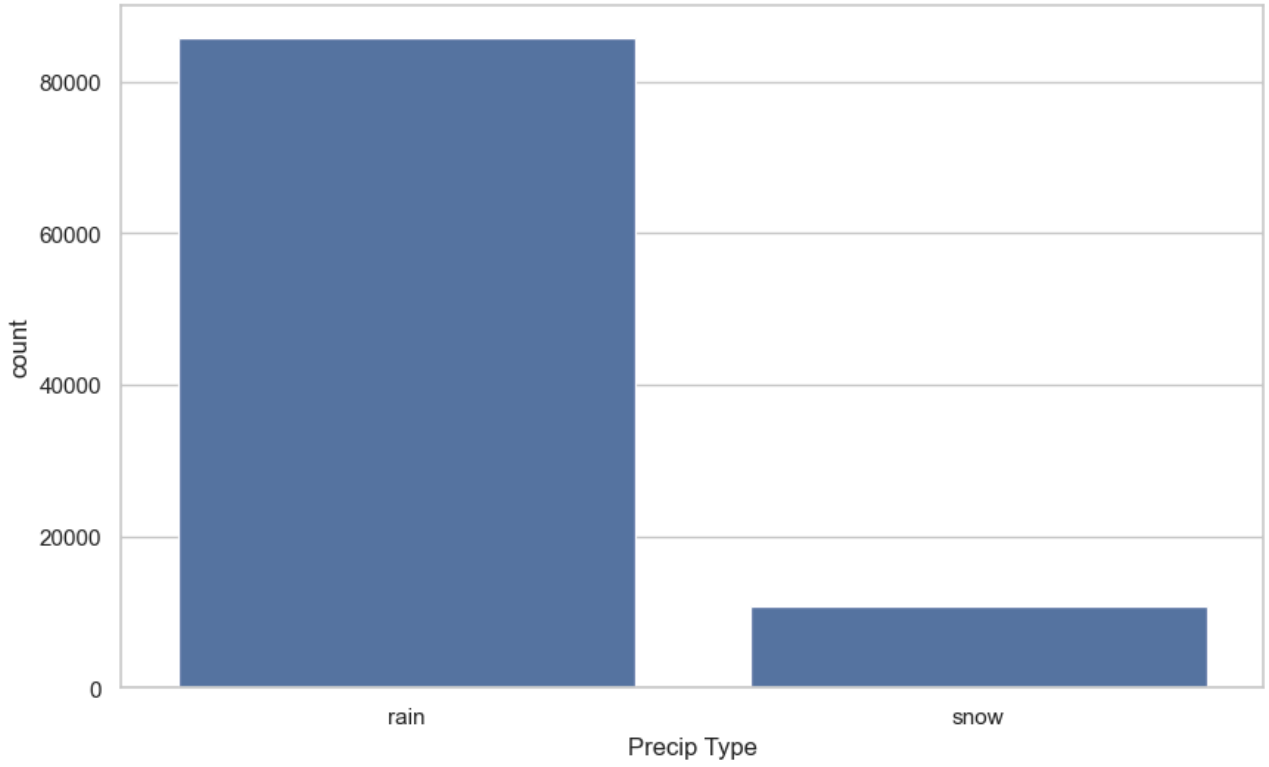
```
Unique weather Summaries: 27
```

Total observations: 96,429. Data is highly imbalanced across weather conditions.

Dominant Weather Conditions Top weather states: Partly Cloudy → 31,726 (~33%) Mostly Cloudy → 28,094 (~29%) Overcast → 16,597 (~17%) Clear → 10,873 (~11%) Foggy → 7,148 (~7%) These 5 categories alone cover ~95%+ of data The weather summary distribution is highly skewed toward stable atmospheric conditions such as partly cloudy, mostly cloudy, and overcast. Rare weather events like rain, drizzle, and windy conditions appear infrequently, indicating a strong class imbalance in weather patterns.

Precip Type

In [31…
```
sns.countplot(x=df['Precip Type'])
plt.show()
```



In [32…
```
df['Precip Type'].value_counts()
```

Out[32]:
```
Precip Type
rain    85717
snow    10712
Name: count, dtype: int64
```

Precipitation in this dataset is heavily rain-dominated.Snow is present but

relatively rare, indicating seasonal or climate-specific variation

Daily Summary

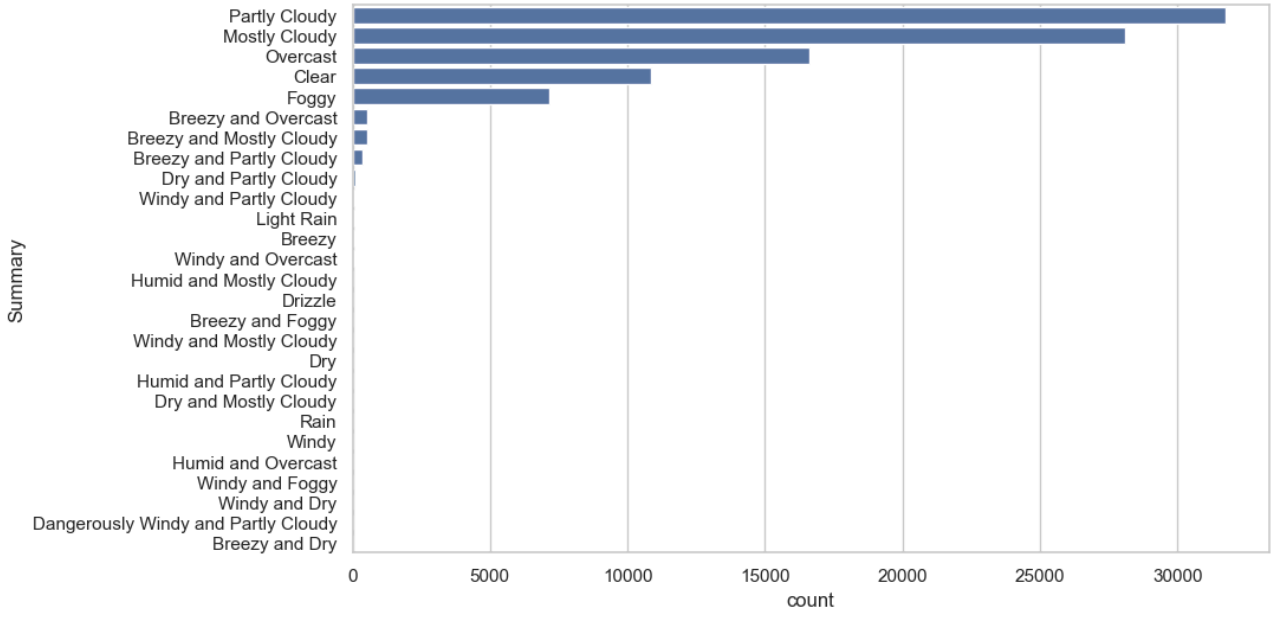
In [33…
```
df['Daily Summary'].value_counts().head()
```

Out[33]:
```
Daily Summary
Mostly cloudy throughout the day.          20085
Partly cloudy throughout the day.           9981
Partly cloudy until night.                  6169
Partly cloudy starting in the morning.      5184
Foggy in the morning.                       4201
Name: count, dtype: int64
```

In [34…
```
df['Daily Summary'].nunique()
```

Out[34]:
```
214
```

In [35…
```
sns.countplot(y=df['Summary'], order=df['Summary'].value_counts().index)
plt.show()
```



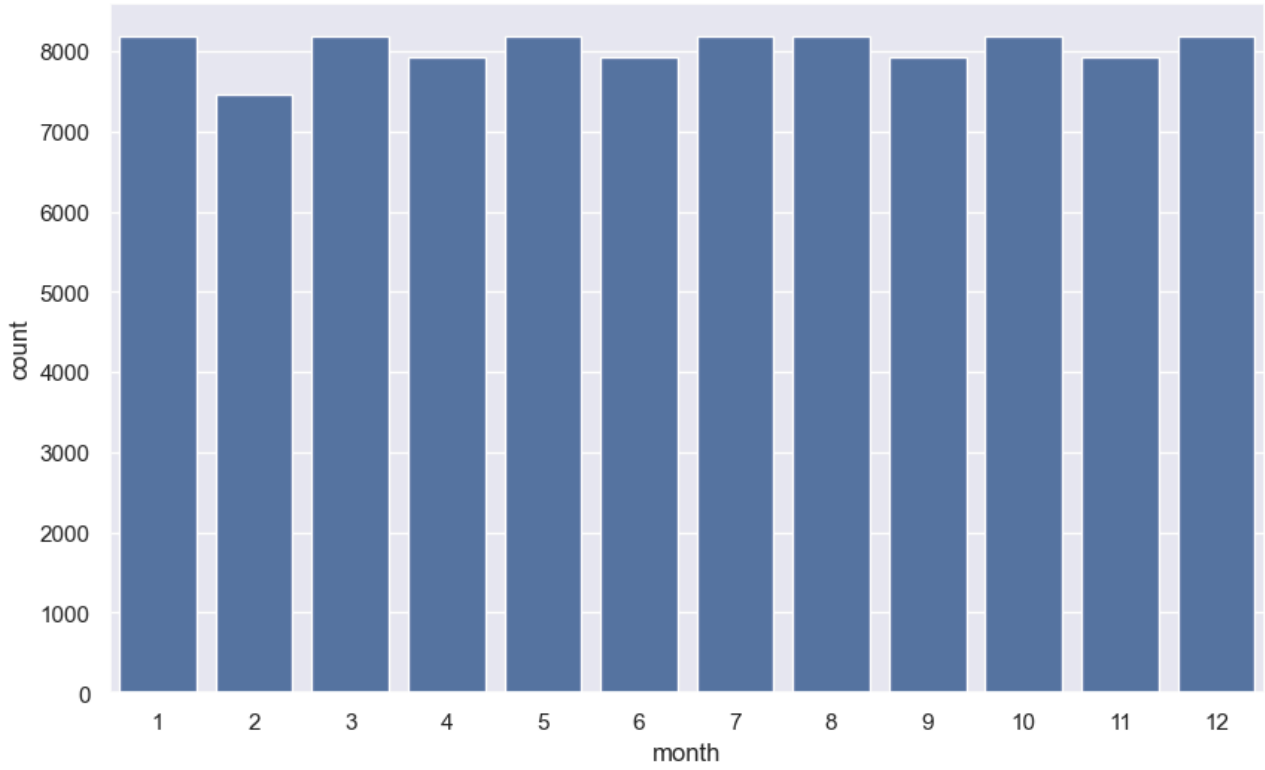
The dataset is strongly skewed toward cloud-dominated daily weather pattes. Weather is often stable or slowly changing rather than extreme. Fog appears as a secondary recurring morning condition

Month (time feature)

In [36…
```
sns.set_style()
```

```
sns.set_theme(style='darkgrid')
sns.countplot(x='month',data=df)
```

Out[36]:
```
<Axes: xlabel='month', ylabel='count'>
```



In [37…
```
df['month'].value_counts()
```

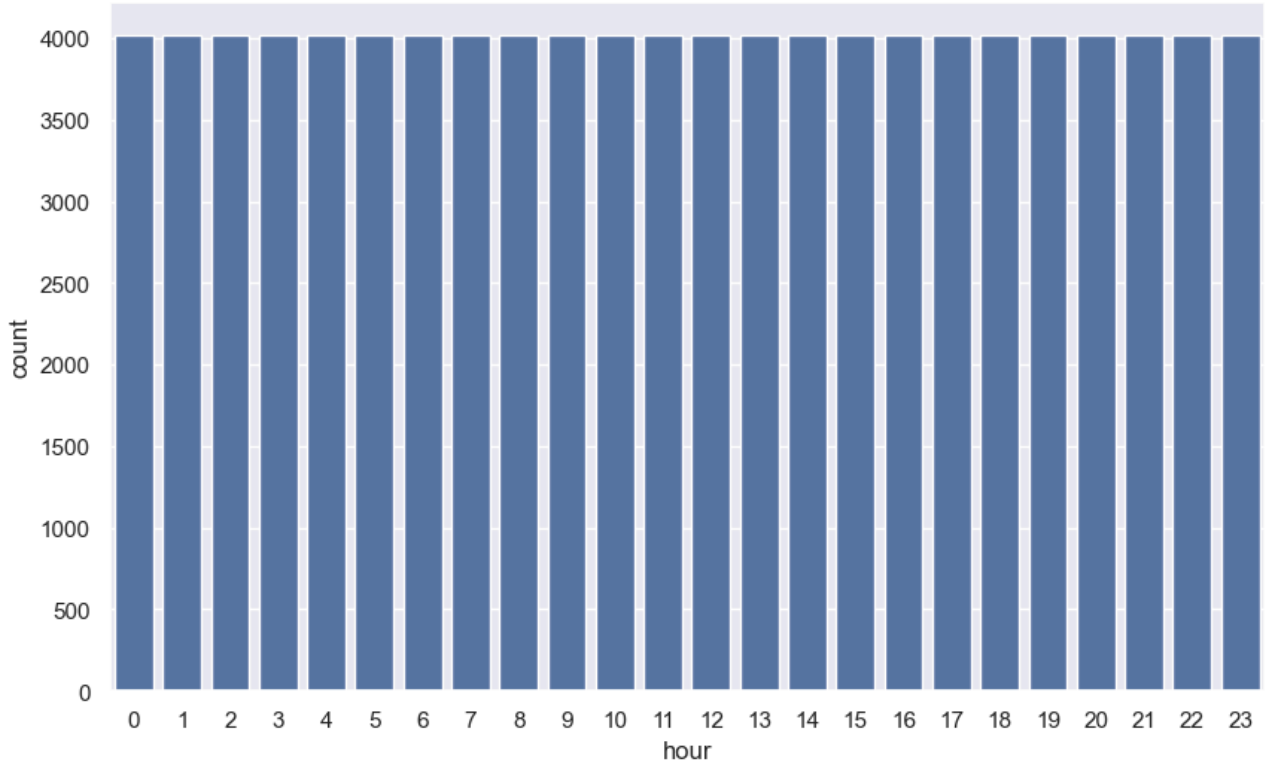
Out[37]:
```
month
3     8184
8     8184
12    8184
1     8184
5     8184
10    8184
7     8181
4     7920
11    7920
6     7920
9     7920
2     7464
Name: count, dtype: int64
```

Monthly distribution is mostly balanced with minor variation. February has fewer records, likely due to fewer calendar day

Hour

In [38…
```
sns.countplot(x='hour',data=df)
```

Out[38]:
```
<Axes: xlabel='hour', ylabel='count'>
```



In [35…
```
df['hour'].value_counts()
```

Out[35]:
```
hour
10    4018
11    4018
20    4018
19    4018
18    4018
17    4018
16    4018
15    4018
14    4018
13    4018
12    4018
21    4018
9     4018
8     4018
7     4018
6     4018
5     4018
4     4018
3     4018
2     4018
1     4018
23    4017
0     4017
22    4017
Name: count, dtype: int64
```

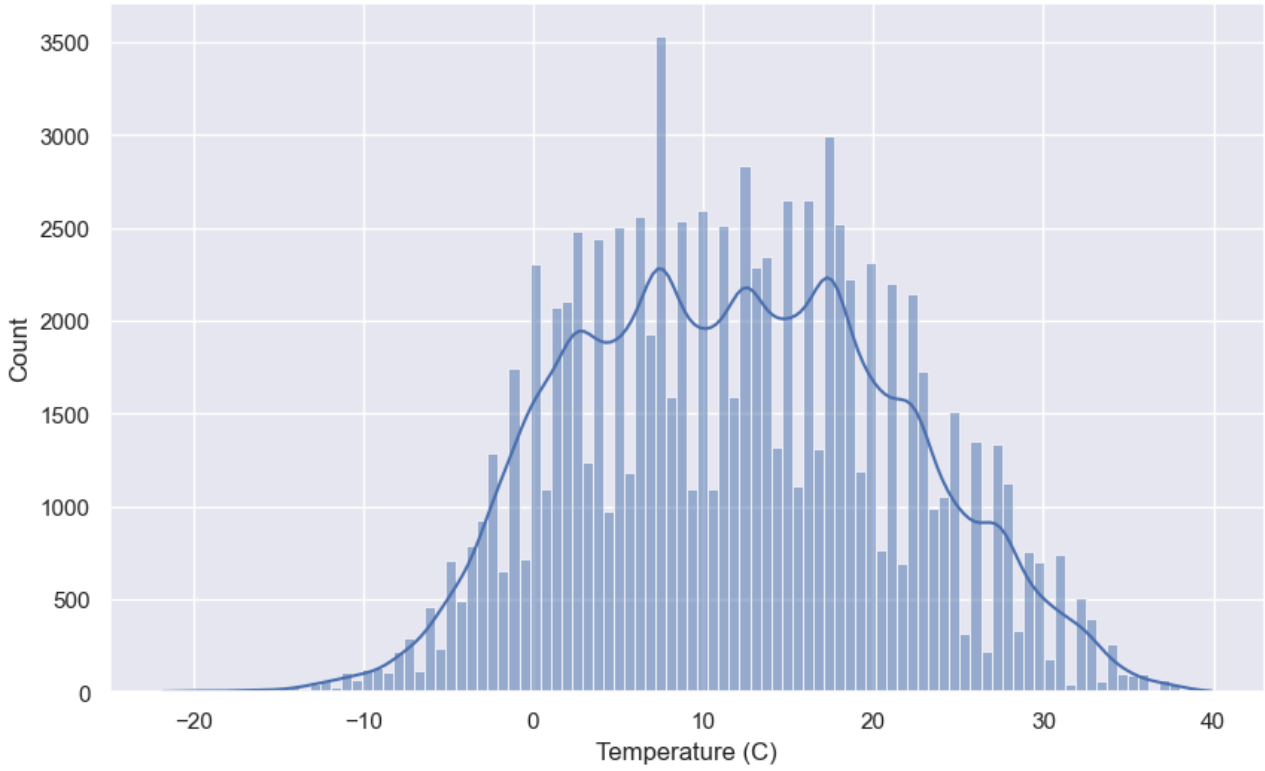
The hourly data is highly balanced

Analysis numerical columns

Temperature (C)

In [39…
```
sns.histplot(x='Temperature (C)',data=df,kde=True)
```

Out[39]:
```
<Axes: xlabel='Temperature (C)', ylabel='Count'>
```



In [37…
```
df['Temperature (C)'].describe()
```

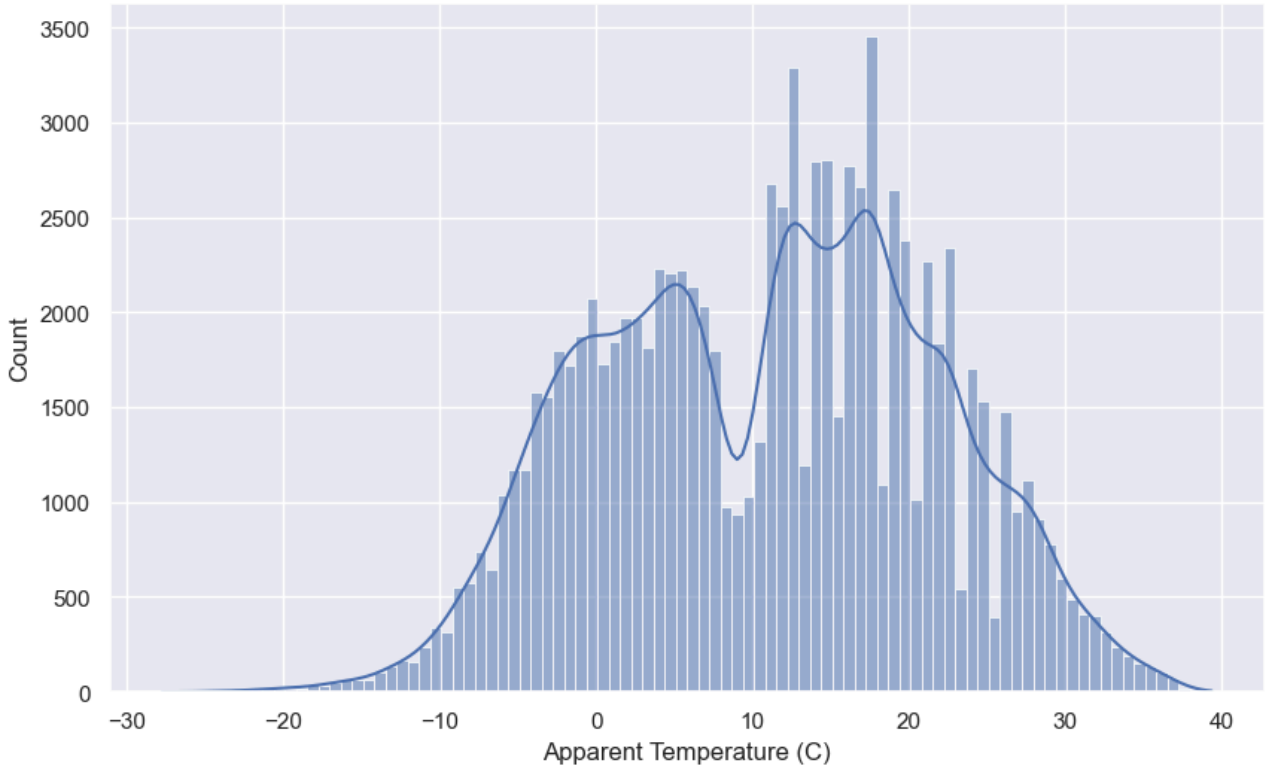
Out[37]:
```
count    96429.000000
mean        11.929692
std          9.550492
min        -21.822222
25%          4.683333
50%         12.000000
75%         18.838889
max         39.905556
Name: Temperature (C), dtype: float64
```

The dataset represents a well-distributed seasonal climate pattern, with most observations in the mild range, but also capturing extreme winter and summer conditions, making it suitable for comprehensive weather analysis.

Apparent Temperature (C)

In [38…
```
sns.histplot(x='Apparent Temperature (C)',data=df,kde=True)
```

Out[38]:
```
<Axes: xlabel='Apparent Temperature (C)', ylabel='Count'>
```



In [39…
```
df['Apparent Temperature (C)'].describe()
```

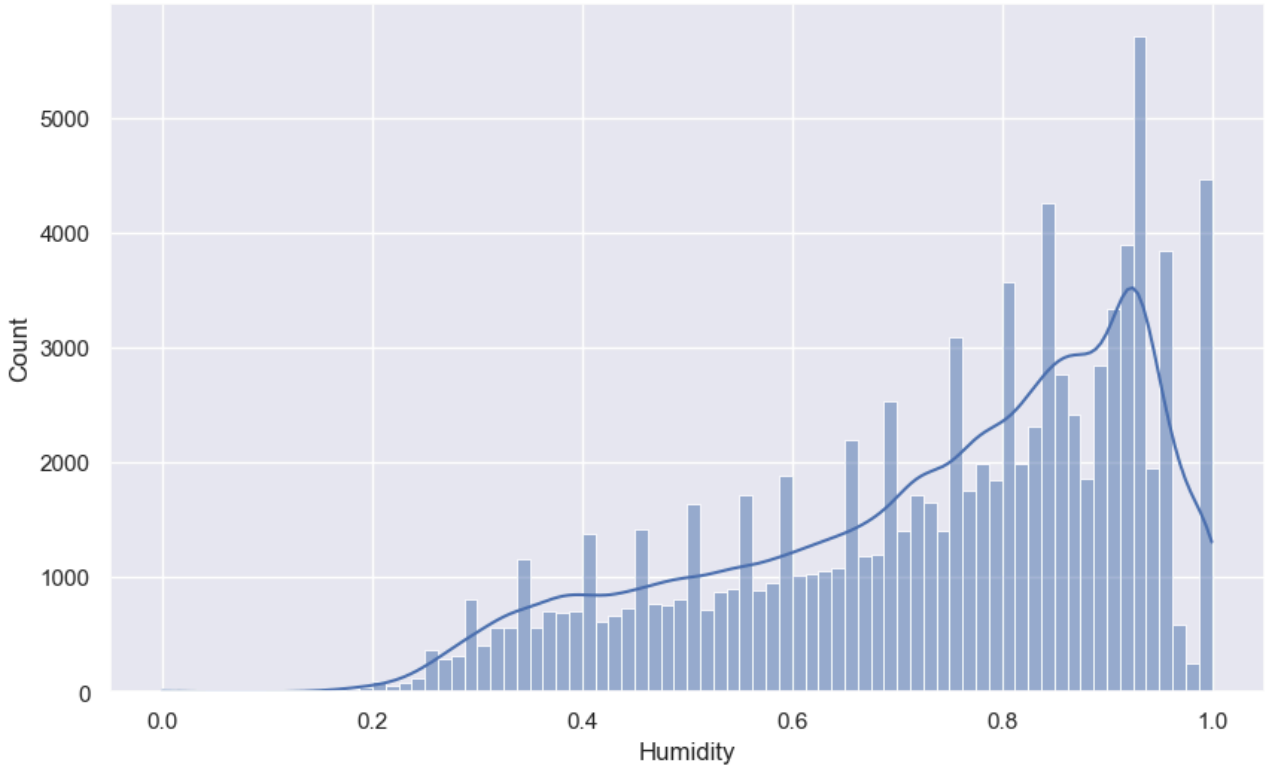
Out[39]:
```
count    96429.000000
mean        10.851707
std         10.695743
min        -27.716667
25%          2.311111
50%         12.000000
75%         18.838889
max         39.344444
Name: Apparent Temperature (C), dtype: float64
```

The dataset shows that the feels-like temperature usually stays mild, but it can sometimes go to very cold or very hot levels. This means the weather changes a lot and is useful for studying how people actually experience temperature.

Humidity

In [40…
```
sns.histplot(x='Humidity',data=df,kde=True)
```

Out[40]:
```
<Axes: xlabel='Humidity', ylabel='Count'>
```



In [41…
```
df['Humidity'].describe()
```

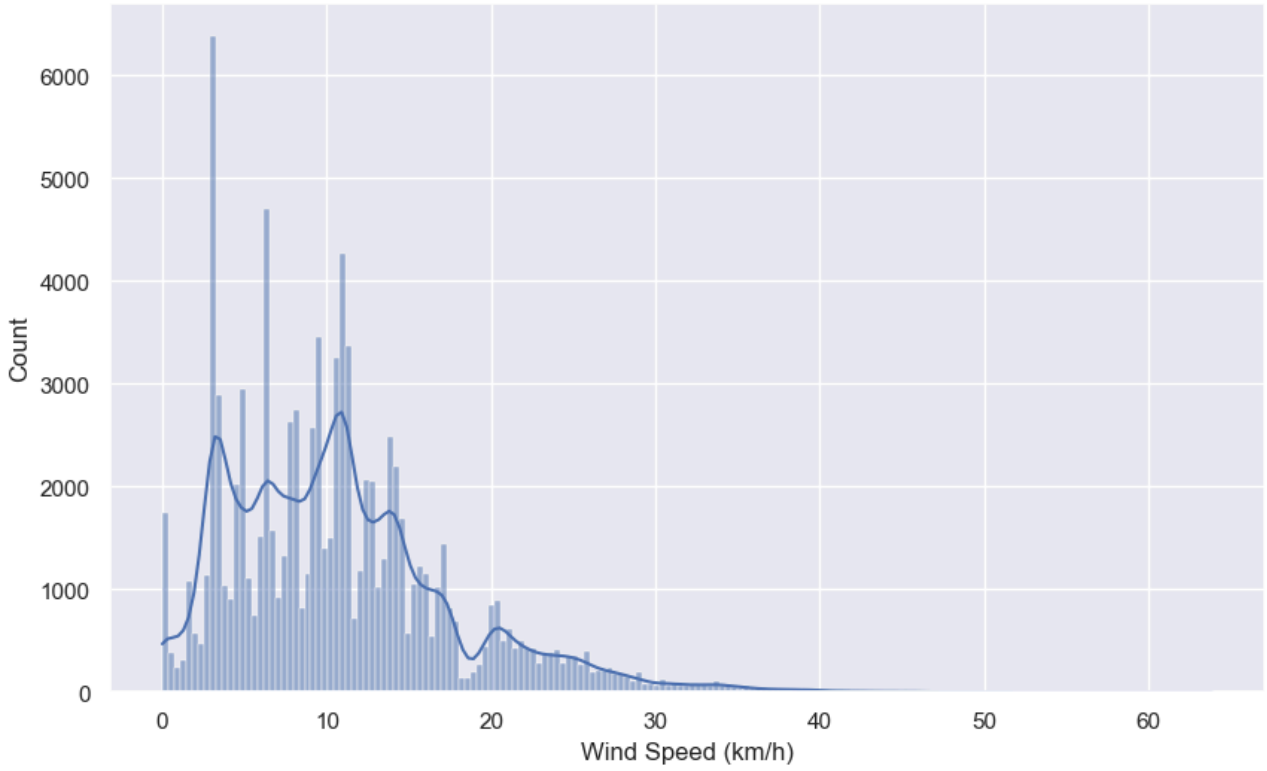
Out[41]:
```
count    96429.000000
mean         0.734902
std          0.195466
min          0.000000
25%          0.600000
50%          0.780000
75%          0.890000
max          1.000000
Name: Humidity, dtype: float64
```

Humidity is mostly in the moderate to high range (0.6 to 0.89).Very low humidity conditions are rare. Overall, the dataset shows that the environment is generally moderately to highly humid, with occasional dry and fully humid conditions.

Wind Speed (km/h)

In [42…
```
sns.histplot(x='Wind Speed (km/h)',data=df,kde=True)
```

Out[42]:
```
<Axes: xlabel='Wind Speed (km/h)', ylabel='Count'>
```



In [43…
```
df['Wind Speed (km/h)'].describe()
```

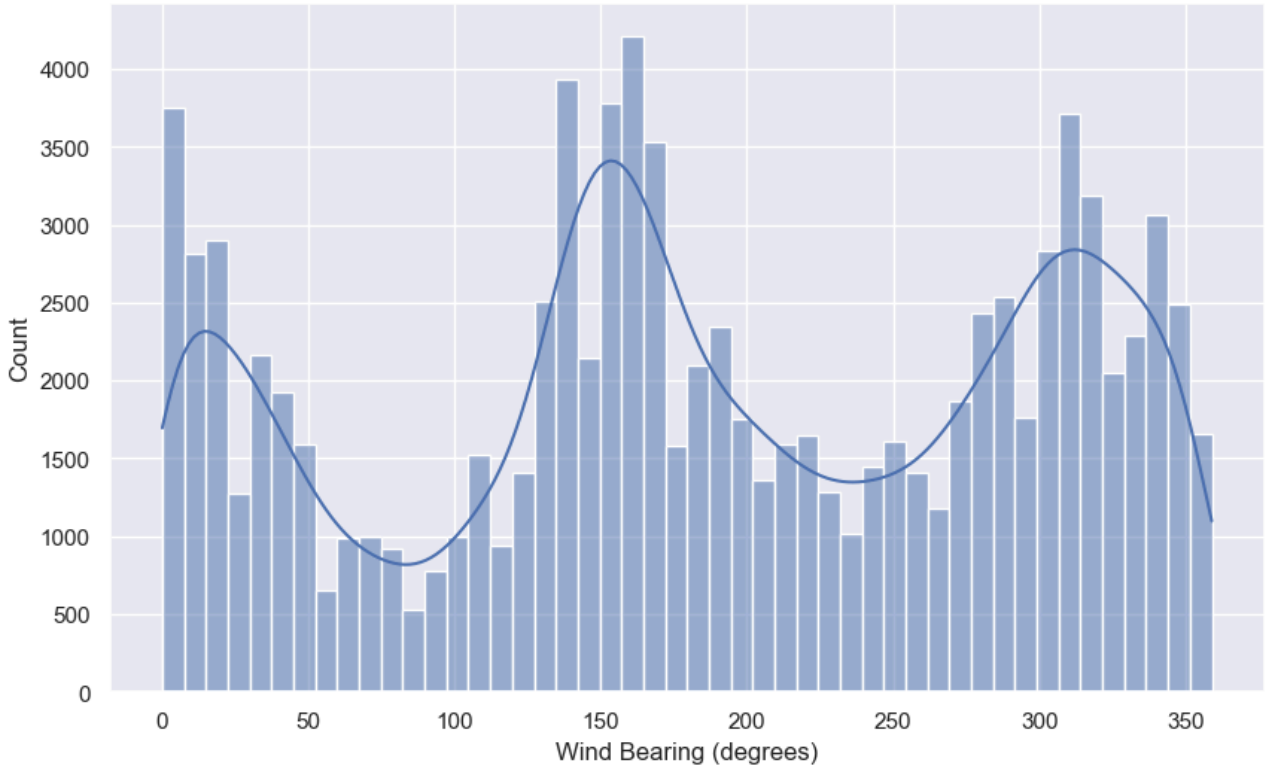
Out[43]:
```
count    96429.000000
mean        10.812460
std          6.913345
min          0.000000
25%          5.828200
50%          9.965900
75%         14.135800
max         63.852600
Name: Wind Speed (km/h), dtype: float64
```

Most wind speeds lie between 5.83 km/h and 14.13 km/h, showing stable and moderate wind conditions.Very high wind speeds are rare but present, indicating occasional extreme weather events.Overall, the dataset represents mostly calm to moderate wind conditions, with a few high wind outliers, making it useful for analyzing both normal and extreme wind patterns.

Wind Bearing (degrees)

In [44…
```
sns.histplot(x='Wind Bearing (degrees)',data=df,kde=True)
```

Out[44]:
```
<Axes: xlabel='Wind Bearing (degrees)', ylabel='Count'>
```



In [45…
```
df['Wind Bearing (degrees)'].describe()
```

Out[45]:
```
count    96429.000000
mean       187.497506
std        107.376423
min          0.000000
25%        116.000000
50%        180.000000
75%        290.000000
max        359.000000
Name: Wind Bearing (degrees), dtype: float64
```

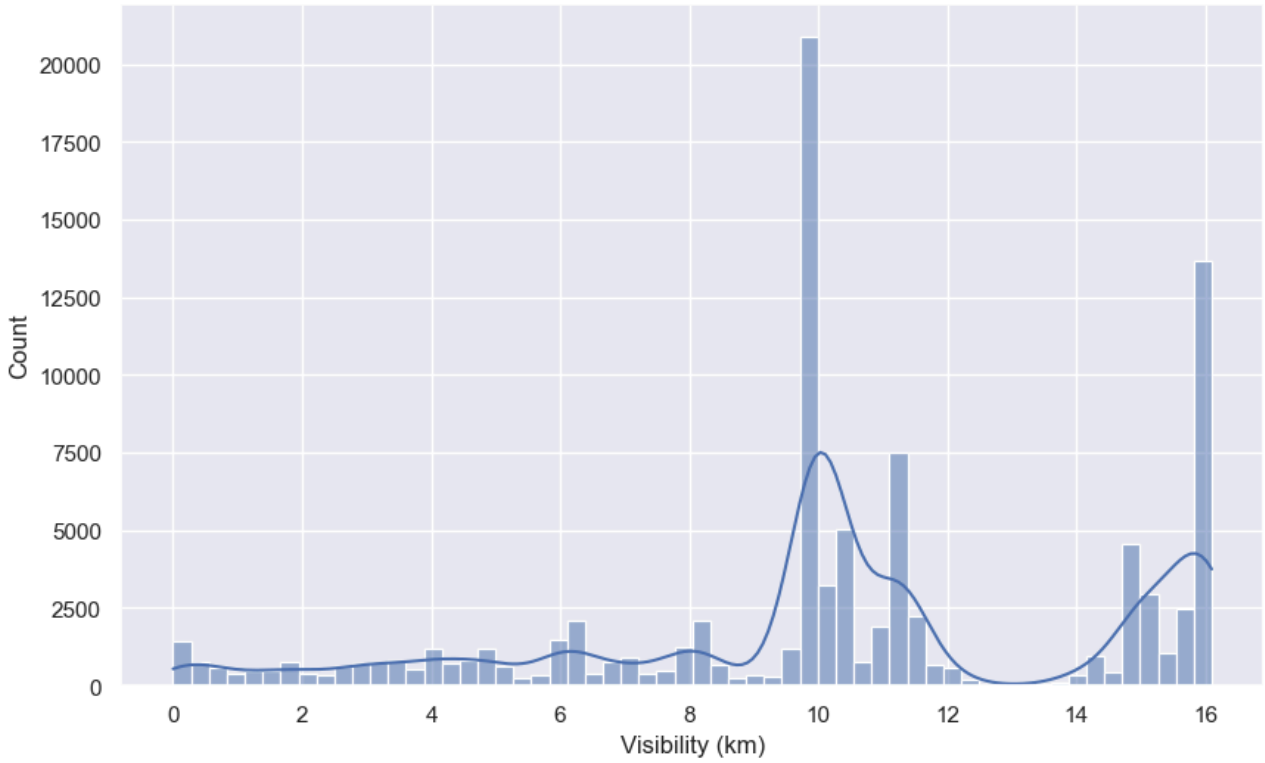
The Wind Bearing (degrees) feature represents wind direction on a circular scale ranging from 0° to 359°.Both mean and median are close to 180° (South).This suggests a slight central tendency toward southern wind directions.The small difference between mean and median indicates fairly balanced distribution around the center.

Wind direction in this dataset is highly diverse and spread across all compass directions, with a mild tendency toward southern directions around 180°, but no strong or dominant wind direction pattern is present.

Visibility (km)

In [46…
```
sns.histplot(x='Visibility (km)',data=df,kde=True)
```

Out[46]: <Axes: xlabel='Visibility (km)', ylabel='Count'>



In [47…
```
df['Visibility (km)'].describe()
```

Out[47]:
```
count    96429.000000
mean        10.347225
std          4.192548
min          0.000000
25%          8.339800
50%         10.046400
75%         14.812000
max         16.100000
Name: Visibility (km), dtype: float64
```

0 km suggests extreme fog or near-zero visibility conditions. 16.1 km indicates very clear atmospheric conditions.Wide range shows presence of both extreme and normal conditions.Atmospheric clarity is moderate and stable most of the time.

isibility in this dataset is generally moderate (~10 km on average) with balanced variation across conditions. While most observations reflect normal atmospheric clarity, occasional extreme low and high visibility conditions indicate changing weather patterns influenced by environmental factors.

Cloud Cover

In [48…
```
df['Cloud Cover'].describe()
```

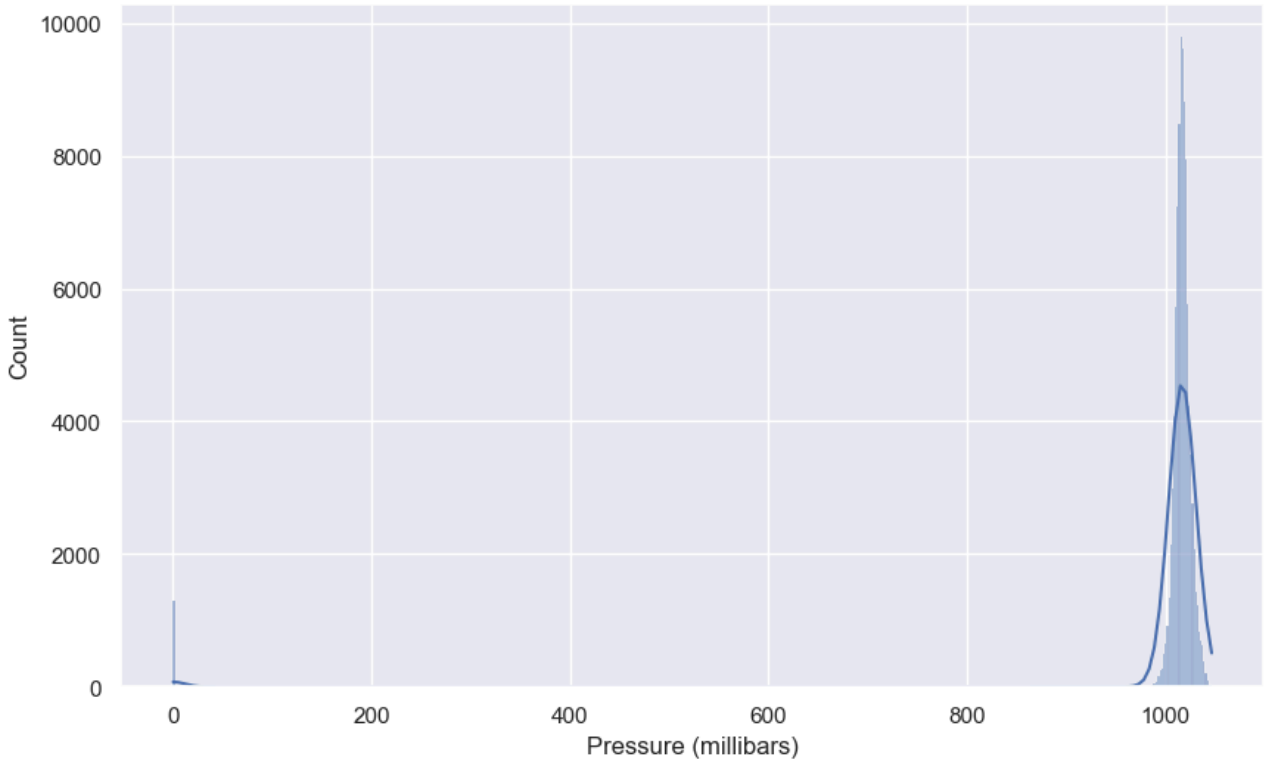
Out[48]:
```
count    96429.0
mean         0.0
std          0.0
min          0.0
25%          0.0
50%          0.0
75%          0.0
max          0.0
Name: Cloud Cover, dtype: float64
```

The Cloud Cover feature contains a single constant value (0.0) across all 96,429 records. As a result, it has no variability and no analytical or predictive value in its current form.

Pressure (millibars)

In [49…
```
sns.histplot(x='Pressure (millibars)',data=df,kde=True)
```

Out[49]:
```
<Axes: xlabel='Pressure (millibars)', ylabel='Count'>
```



In [50…
```
df['Pressure (millibars)'].describe()
```

Out[50]:
```
count    96429.000000
mean      1003.232915
std        116.984300
min          0.000000
25%       1011.900000
50%       1016.450000
75%       1021.090000
max       1046.380000
Name: Pressure (millibars), dtype: float64
```

50% of values lie between 1011.9 and 1021.09 mb.This is a tight and stable range.Indicates most weather conditions occur under normal pressure conditions.

Atmospheric pressure is generally stable around 1010–1020 mb, indicating normal weather conditions dominate the dataset. However, the presence of invalid zero values significantly distorts the distribution.

The value 0 in Pressure (millibars) is not physically possible in real-world atmospheric conditions. Therefore, it is treated as a missing/erroneous value and replaced with NaN to maintain data accuracy during EDA

In [51…
```
df[df['Pressure (millibars)'] == 0].shape[0]
```

Out[51]:
```
1288
```

In [52…
```
df['Pressure (millibars)'] = df['Pressure (millibars)'].replace(0, np.nan

df['Pressure (millibars)'].describe()
```

Out[52]:
```
count    95141.000000
mean      1016.814483
std          7.779265
min        973.780000
25%       1012.120000
50%       1016.550000
75%       1021.170000
max       1046.380000
Name: Pressure (millibars), dtype: float64
```

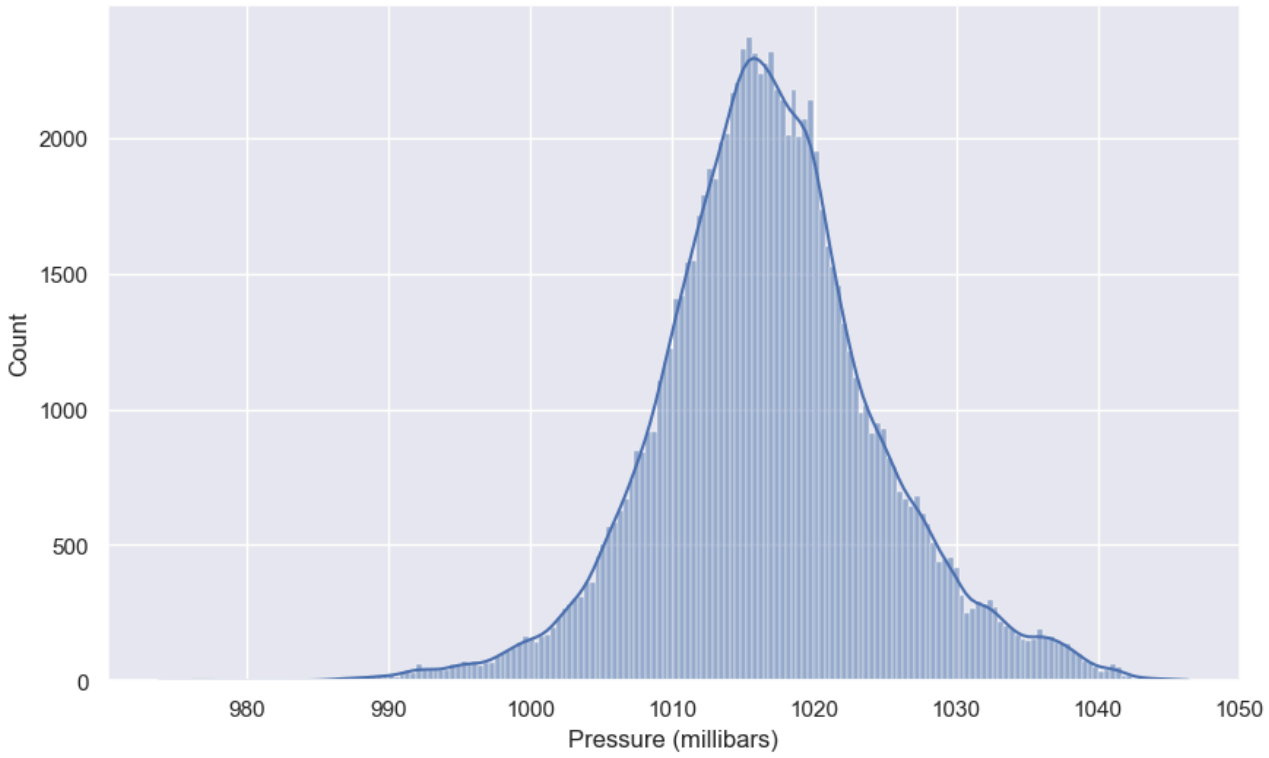
In [53…
```
df['Pressure (millibars)'].isna().sum()
```

Out[53]:
```
np.int64(1288)
```

After cleaning invalid zero values

In [54…
```
sns.histplot(x='Pressure (millibars)',data=df,kde=True)
```

Out[54]:
```
<Axes: xlabel='Pressure (millibars)', ylabel='Count'>
```



In [55…
```
df[df['Pressure (millibars)'] == 0].shape[0]
```

Out[55]:
```
0
```

In [56…
```
df['Pressure (millibars)'].describe()
```

Out[56]:
```
count    95141.000000
mean      1016.814483
std          7.779265
min        973.780000
25%       1012.120000
50%       1016.550000
75%       1021.170000
max       1046.380000
Name: Pressure (millibars), dtype: float64
```

Pressure is a stable atmospheric feature with moderate variation.

After cleaning invalid zero values, atmospheric pressure shows a realistic and stable distribution centered around ~1016 mb, with moderate natural variation. The feature is now physically meaningful, statistically reliable, and highly useful for weather analysis

Final Univariate Analysis Summary

The univariate analysis of the weather dataset was performed to understand the distribution and behavior of each individual feature, including numerical, categorical, and time-based variables. Overall, the dataset shows a mix of stable climatic variables and a few highly imbalanced weather event features.

Overall Insight

The dataset is well-structured and suitable for further analysis. It contains:
Strong numerical features with meaningful variation

Highly imbalanced categorical weather conditions

Clear temporal patterns across hours, months

One weak feature (cloud cover) with minimal variance

Final Conclusion

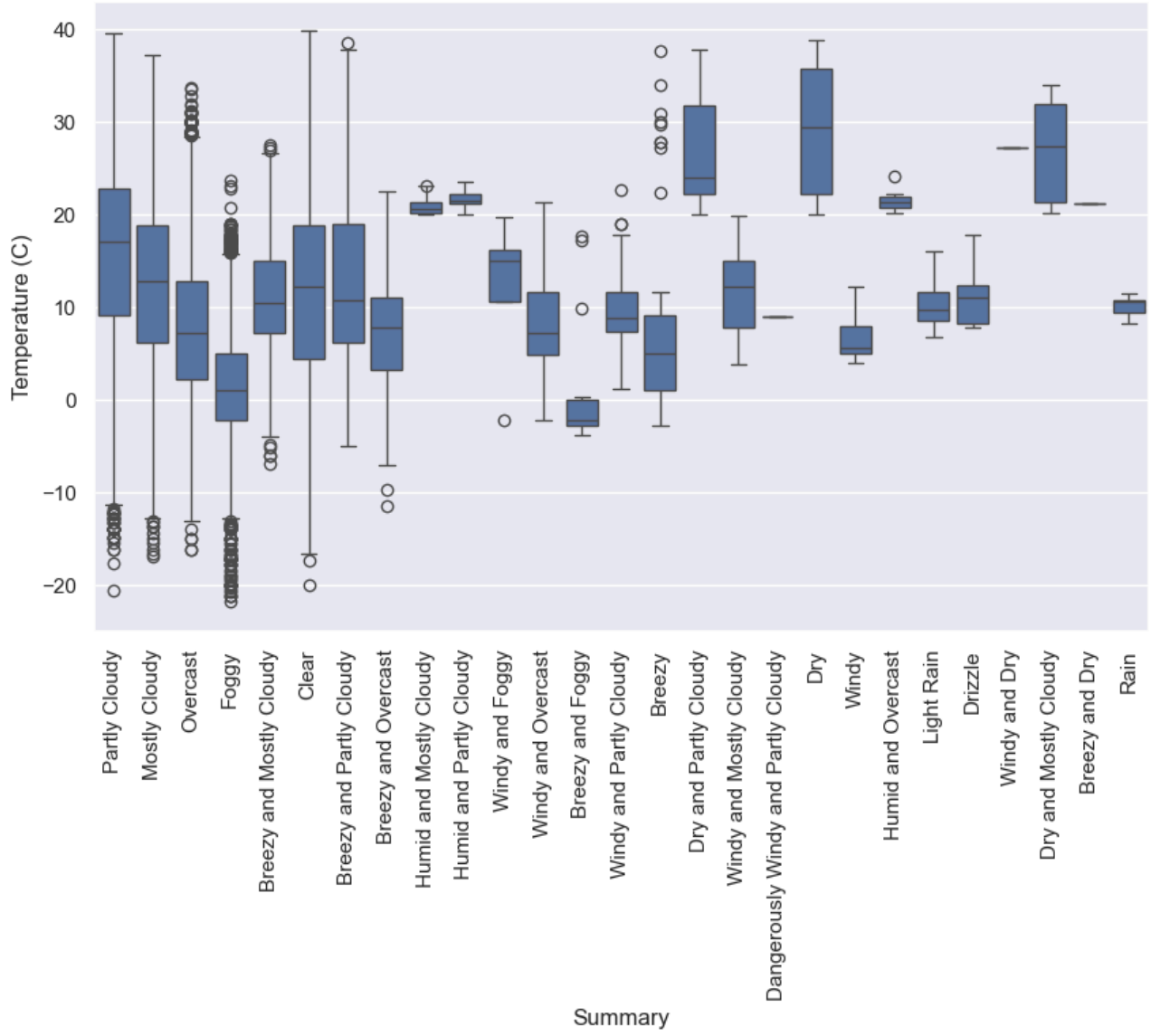
The univariate analysis reveals that the weather dataset is dominated by stable atmospheric conditions with clear seasonal and temporal patterns. While numerical features show meaningful variation, categorical variables are highly imbalanced, reflecting real-world weather dominance of calm and cloudy conditions.

BIVARIATE ANALYSIS

The purpose of bivariate analysis is to understand the relationship between temperature and other atmospheric as well as time-based variables. Through this analysis, we aim to identify how weather conditions such as humidity, pressure, wind speed, visibility, and seasonal patterns influence temperature behavior. This helps reveal meaningful environmental patterns, daily weather cycles, and seasonal climatic variations present in the dataset

Temperature (C) vs Summary

In [57…
```
sns.boxplot(x='Summary',y='Temperature (C)',data=df)
plt.xticks(rotation=90)
plt.show()
```



In [58…

```
df.groupby('Summary')['Temperature (C)'].agg(['mean','min','max','std'])
```

Out[58]:

Summary	mean	min	max	std
Breezy	7.922016	-2.777778	37.588889	10.706180
Breezy and Dry	21.111111	21.111111	21.111111	NaN
Breezy and Foggy	-0.510317	-3.827778	17.616667	5.081695
Breezy and Mostly Cloudy	11.093411	-6.966667	27.522222	6.683639
Breezy and Overcast	7.241614	-11.533333	22.433333	5.592556
Breezy and Partly Cloudy	12.492761	-5.050000	38.438889	8.140788
Clear	11.909478	-20.000000	39.905556	10.017300
Dangerously Windy and Partly Cloudy	8.944444	8.944444	8.944444	NaN
Drizzle	10.847578	7.805556	17.844444	2.485500
Dry	29.083660	20.000000	38.750000	6.797474
Dry and Mostly Cloudy	26.838492	20.072222	33.905556	5.383821
Dry and Partly Cloudy	26.605749	20.000000	37.800000	5.634725
Foggy	1.464035	-21.822222	23.661111	5.756860
Humid and Mostly Cloudy	20.886389	20.000000	23.111111	0.880749
Humid and Overcast	21.515079	20.105556	24.055556	1.322571
Humid and Partly Cloudy	21.568301	20.055556	23.566667	0.899273
Light Rain	10.021517	6.811111	16.011111	2.063951
Mostly Cloudy	12.629334	-16.894444	37.155556	8.454565
Overcast	7.516502	-16.111111	33.644444	7.112253
Partly Cloudy	16.021960	-20.555556	39.588889	9.446230
Rain	10.096111	8.205556	11.500000	1.061116
Windy	6.804861	3.938889	12.172222	2.870525
Windy and Dry	27.222222	27.222222	27.222222	NaN
Windy and Foggy	11.876389	-2.150000	19.633333	9.601677
Windy and Mostly Cloudy	11.834603	3.888889	19.905556	3.961356
Windy and Overcast	7.932963	-2.266667	21.305556	5.411017
Windy and Partly Cloudy	9.968076	1.111111	22.705556	4.522439

This bivariate analysis shows how temperature changes across different weather conditions.

Dry weather conditions have the highest temperatures, averaging around

26–29°C, indicating hot and dry climates.

Humid conditions also show relatively high and stable temperatures around 20–21°C. In contrast, foggy and overcast conditions are associated with lower temperatures, with foggy weather averaging around 1.5°C and overcast around 7–8°C, indicating colder conditions.

Common weather types such as clear, partly cloudy, and mostly cloudy fall in a moderate temperature range of about 10–16°C, representing typical weather conditions in the dataset.

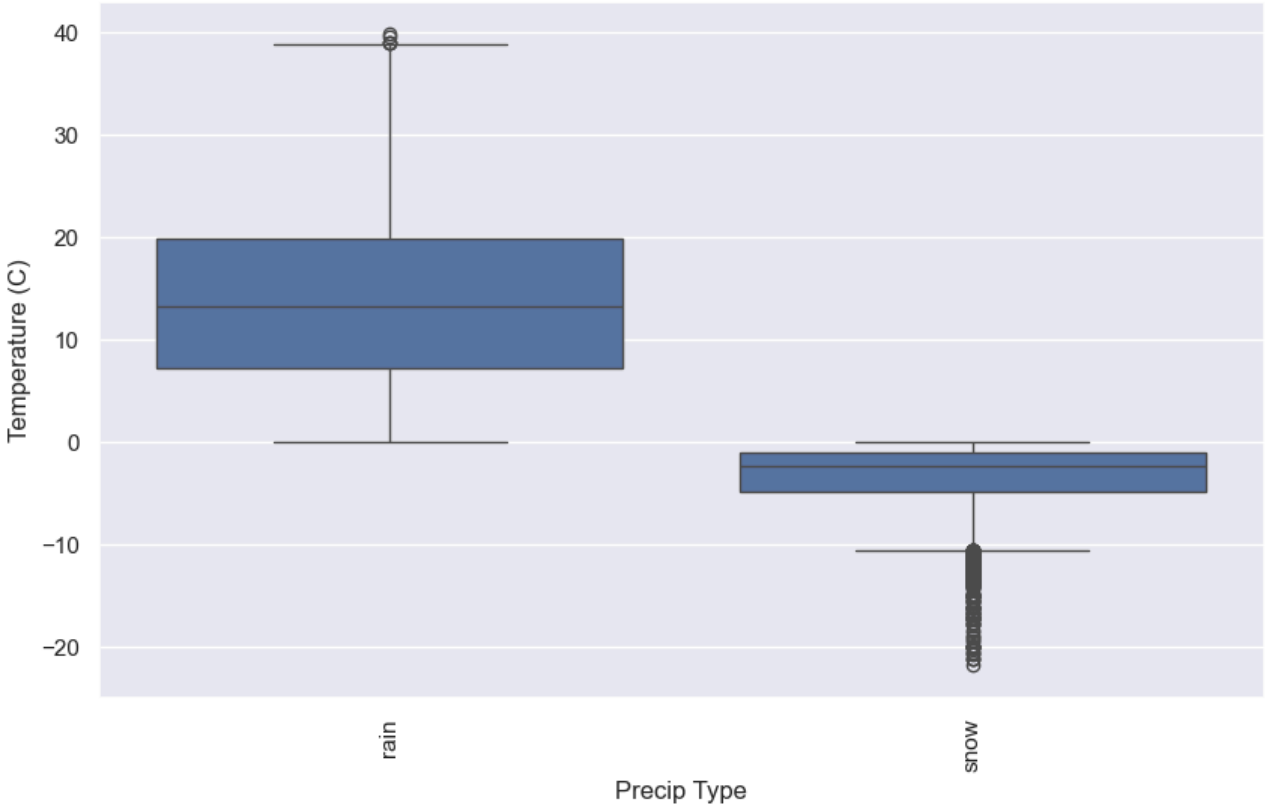
Rain and drizzle conditions also remain in the mild temperature range around 10°C. Windy conditions show mixed behavior, with temperatures varying depending on cloud and fog combinations.

Overall, the analysis clearly shows that dry weather is associated with higher temperatures, foggy and overcast weather with lower temperatures, and cloudy or clear conditions represent the most frequent and moderate climate range

Temperature (C) vs Precip Type

In [59…

```
sns.boxplot(x='Precip Type',y='Temperature (C)',data=df)
plt.xticks(rotation=90)
plt.show()
```



In [60…
```
df.groupby('Precip Type')['Temperature (C)'].agg(['mean','min','max','std
```

Out[60]:

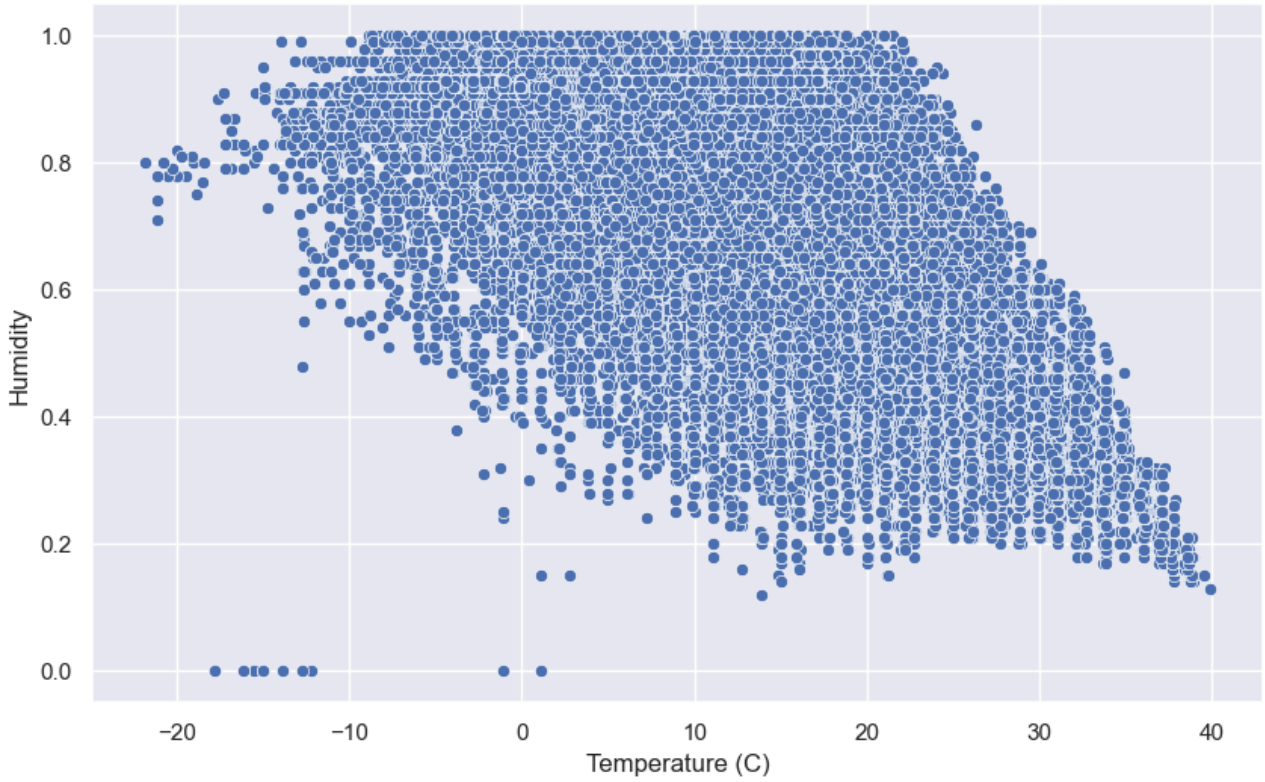
Precip Type	mean	min	max	std
rain	13.829298	0.005556	39.905556	8.303308
snow	-3.270885	-21.822222	0.000000	3.075312

This analysis shows how temperature changes based on precipitation type (rain or snow).This bivariate analysis between precipitation type and temperature shows a clear relationship between the two variables. Rain is associated with higher temperatures, with an average of around 13.8°C and ranging from 0°C to 39.9°C, indicating that rainfall mostly occurs in mild to warm conditions. In contrast, snow is linked with much lower temperatures, with an average of around -3.27°C and values ranging from -21.8°C to 0°C, showing that snowfall occurs only in cold conditions.

Overall, the analysis clearly indicates that warmer temperatures are associated with rain, while colder temperatures are associated with snow, highlighting a strong dependency of precipitation type on temperature.

Temperature (C) vs Humidity

In [61…
```
sns.scatterplot(x='Temperature (C)', y='Humidity',data=df)
plt.show()
```



In [62…
```
df[['Temperature (C)', 'Humidity']].corr()
```

Out[62]:

	Temperature (C)	Humidity
Temperature (C)	1.000000	-0.632331
Humidity	-0.632331	1.000000

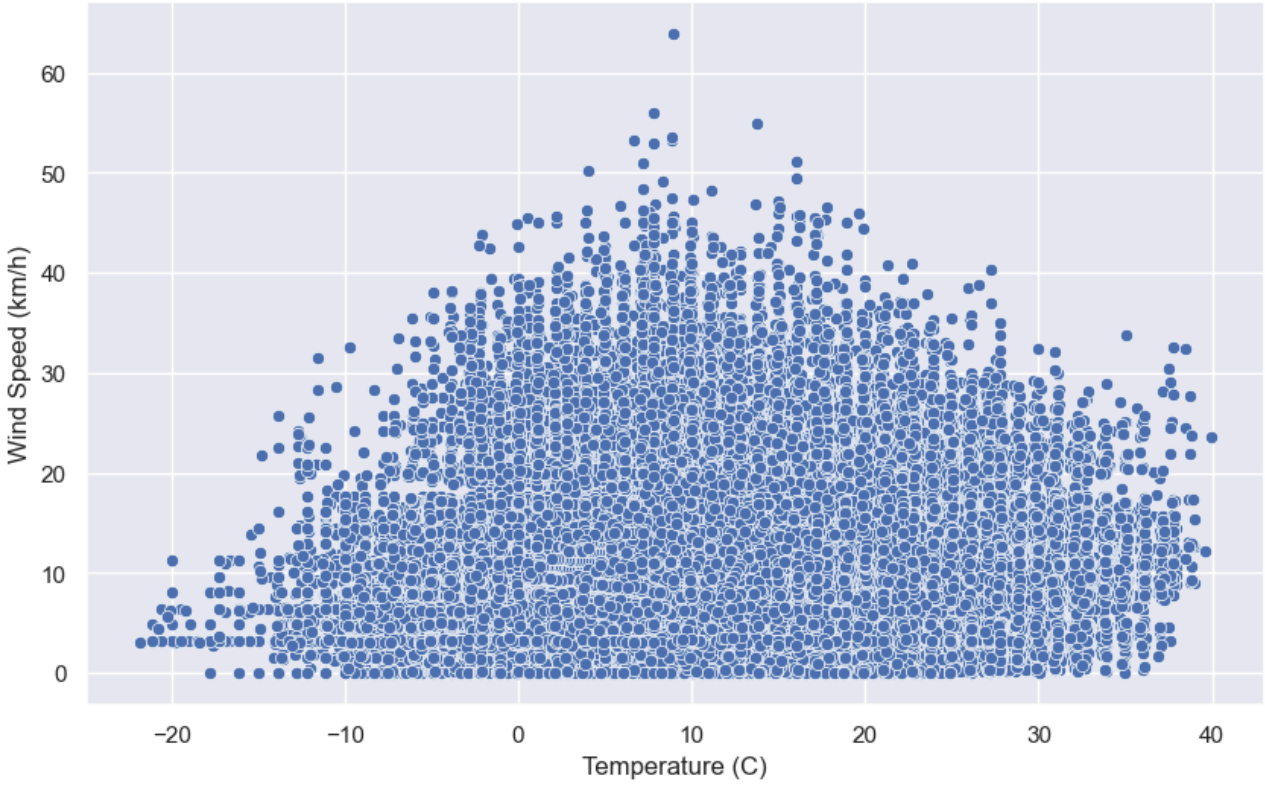
This correlation analysis shows a moderate negative relationship between temperature and humidity. The correlation value of -0.63 indicates that when temperature increases, humidity generally decreases, and when temperature decreases, humidity tends to increase. This reflects a typical weather pattern where warmer air holds less moisture, leading to lower humidity levels, while cooler conditions are associated with higher humidity.

Overall, there is a clear inverse relationship between temperature and humidity in the dataset.

The hypothesis 1 was validated during bivariate analysis using correlation analysis and scatter plots. A strong negative relationship was observed between Temperature and Relative Humidity, indicating that humidity generally decreases as temperature increases.

Temperature (C) vs Wind Speed (km/h)

In [63…
```
sns.scatterplot(x='Temperature (C)', y='Wind Speed (km/h)',data=df)
plt.show()
```



In [64…
```
df[['Temperature (C)', 'Wind Speed (km/h)']].corr()
```

Out[64]:

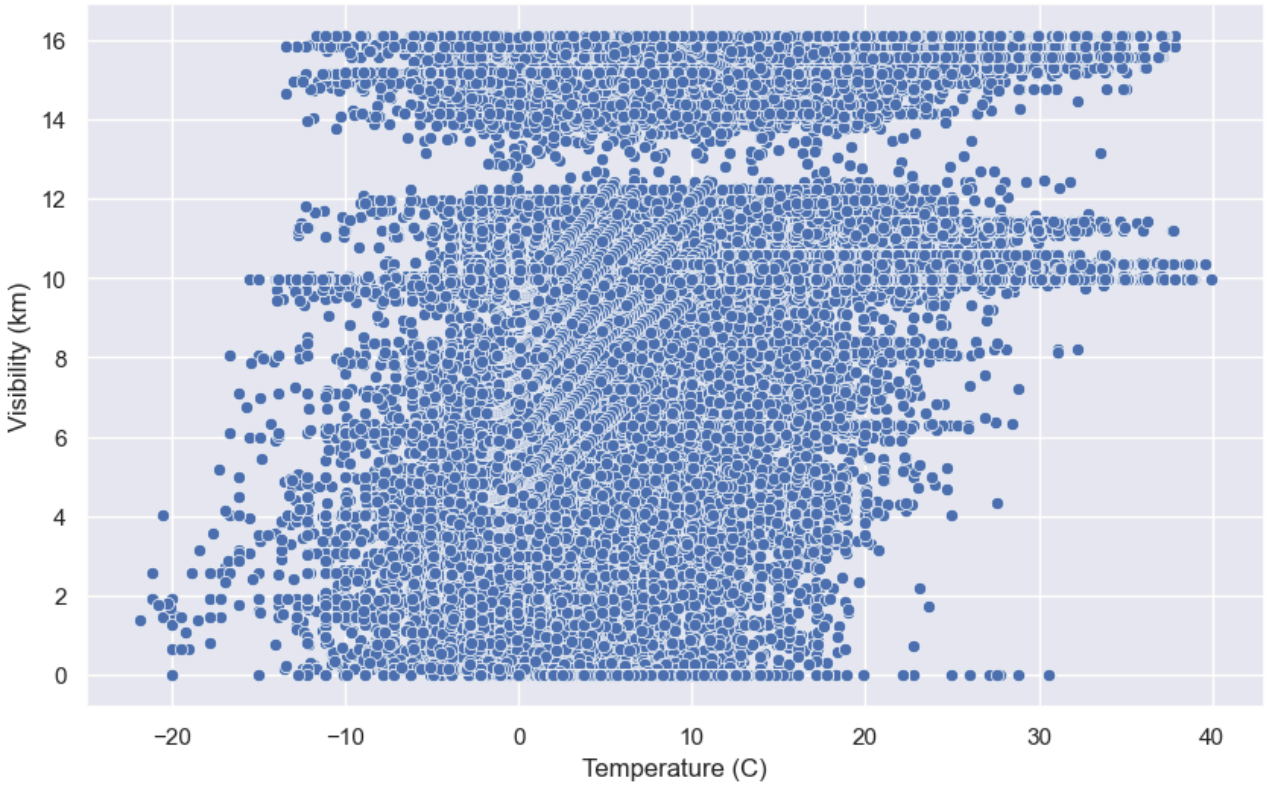
	Temperature (C)	Wind Speed (km/h)
Temperature (C)	1.000000	0.009312
Wind Speed (km/h)	0.009312	1.000000

This correlation analysis shows that there is **almost no relationship** between temperature and wind speed. The correlation value of **0.0093** is very close to zero, which means changes in temperature do not significantly affect wind speed, and vice versa. In simple terms, wind speed and temperature behave independently in this dataset.

Overall, there is **no meaningful correlation** between these two variables.

Temperature (C) vs Visibility (km)

In [65…
```
sns.scatterplot(x='Temperature (C)', y='Visibility (km)',data=df)
plt.show()
```



In [66…
```
df[['Temperature (C)', 'Visibility (km)']].corr()
```

Out[66]:

	Temperature (C)	Visibility (km)
Temperature (C)	1.000000	0.392939
Visibility (km)	0.392939	1.000000

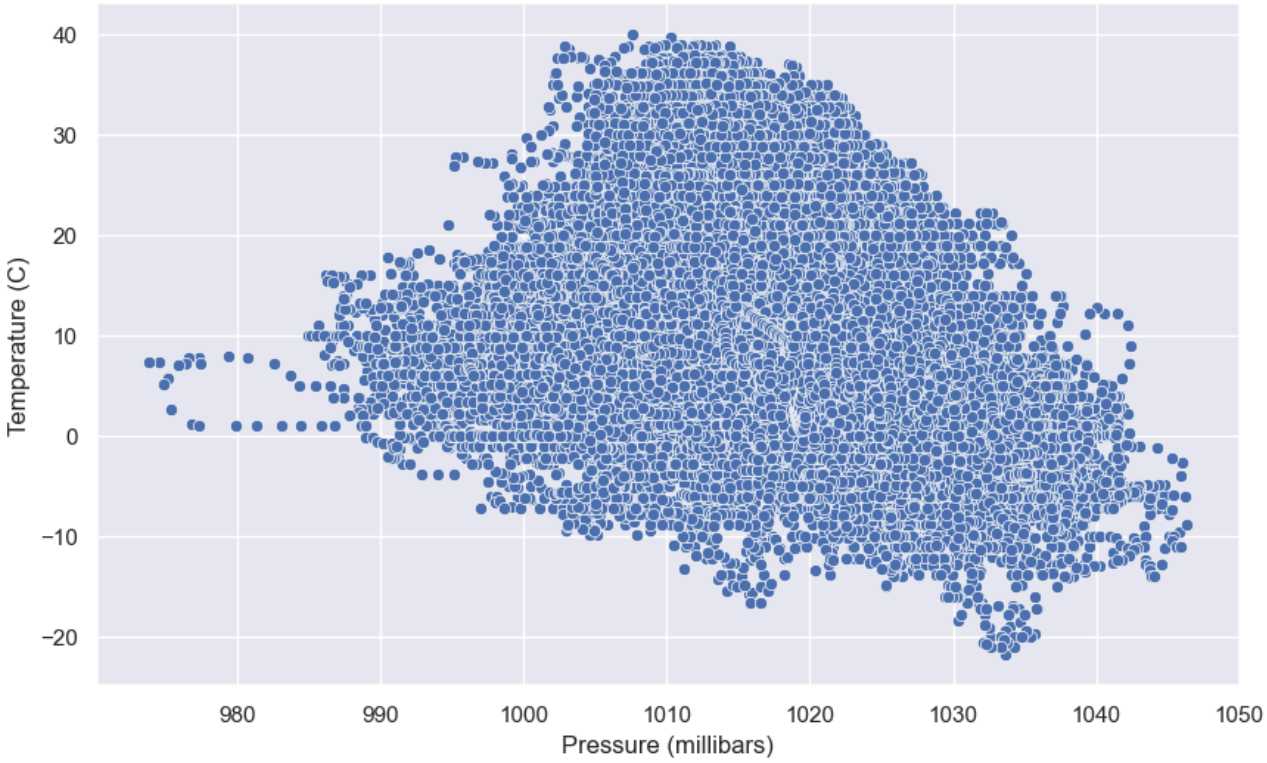
This correlation analysis shows a moderate positive relationship between temperature and visibility. The correlation value of 0.39 indicates that when temperature increases, visibility tends to improve slightly, and when temperature decreases, visibility generally reduces. This suggests that warmer conditions are often associated with clearer air, while colder conditions may have more fog, haze, or moisture in the atmosphere. However, the relationship is not very strong, meaning other weather factors also play an important role. Overall, there is a weak-to-moderate positive

correlation between temperature and visibility.

Temperature (C) vs Pressure (millibars)

In [67…
```
sns.scatterplot(x='Pressure (millibars)',y='Temperature (C)',data=df)
```

Out[67]:
```
<Axes: xlabel='Pressure (millibars)', ylabel='Temperature (C)'>
```



In [68…
```
df[['Temperature (C)', 'Pressure (millibars)']].corr()
```

Out[68]:

	Temperature (C)	Pressure (millibars)
Temperature (C)	1.000000	-0.310469
Pressure (millibars)	-0.310469	1.000000

This correlation analysis shows a **moderate negative relationship** between temperature and pressure. The correlation value of **-0.31** indicates that when temperature increases, air pressure tends to decrease slightly, and when temperature decreases, pressure tends to increase. However, the relationship is not very strong, meaning other weather factors also influence pressure changes.

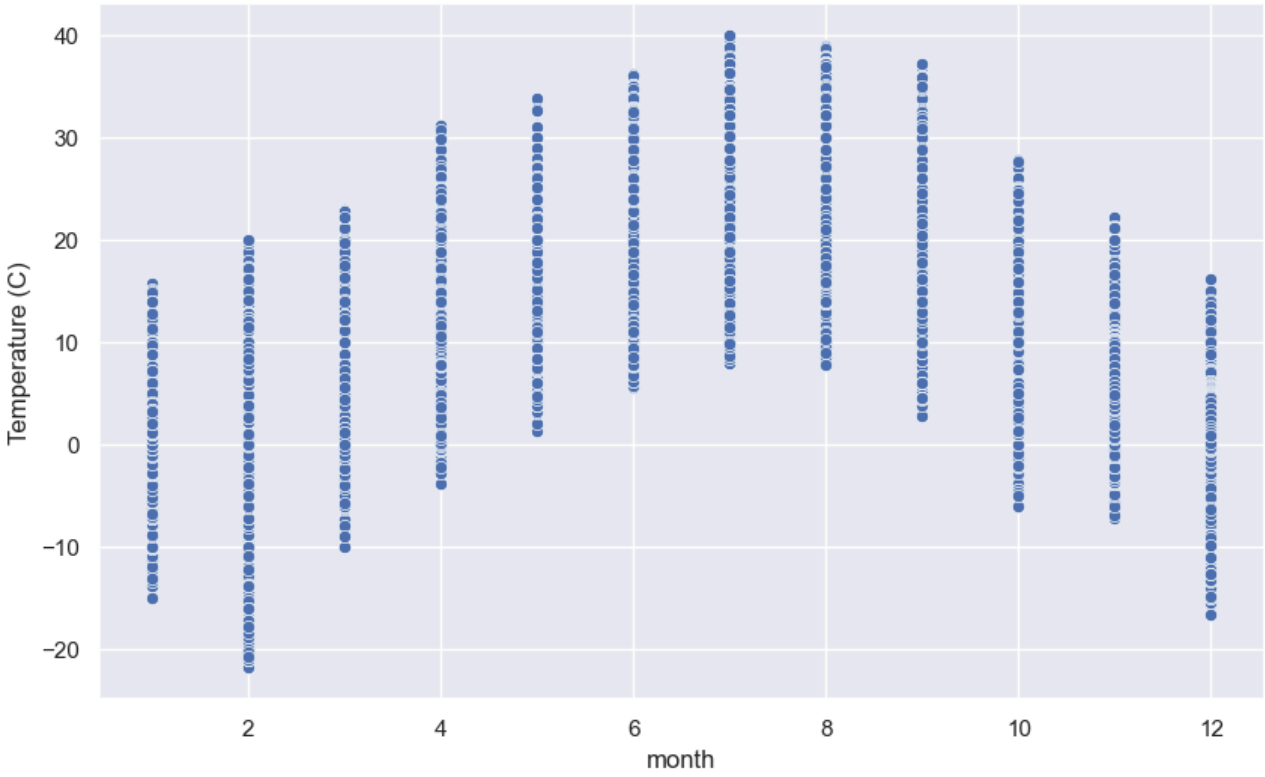
Overall, the dataset shows a **slight inverse relationship between**

temperature and atmospheric pressure.

Temperature (C) vs month

In [69…
```
sns.scatterplot(x='month',y='Temperature (C)',data=df)
```

Out[69]:
```
<Axes: xlabel='month', ylabel='Temperature (C)'>
```



In [70…
```
df[['Temperature (C)', 'month']].corr()
```

Out[70]:

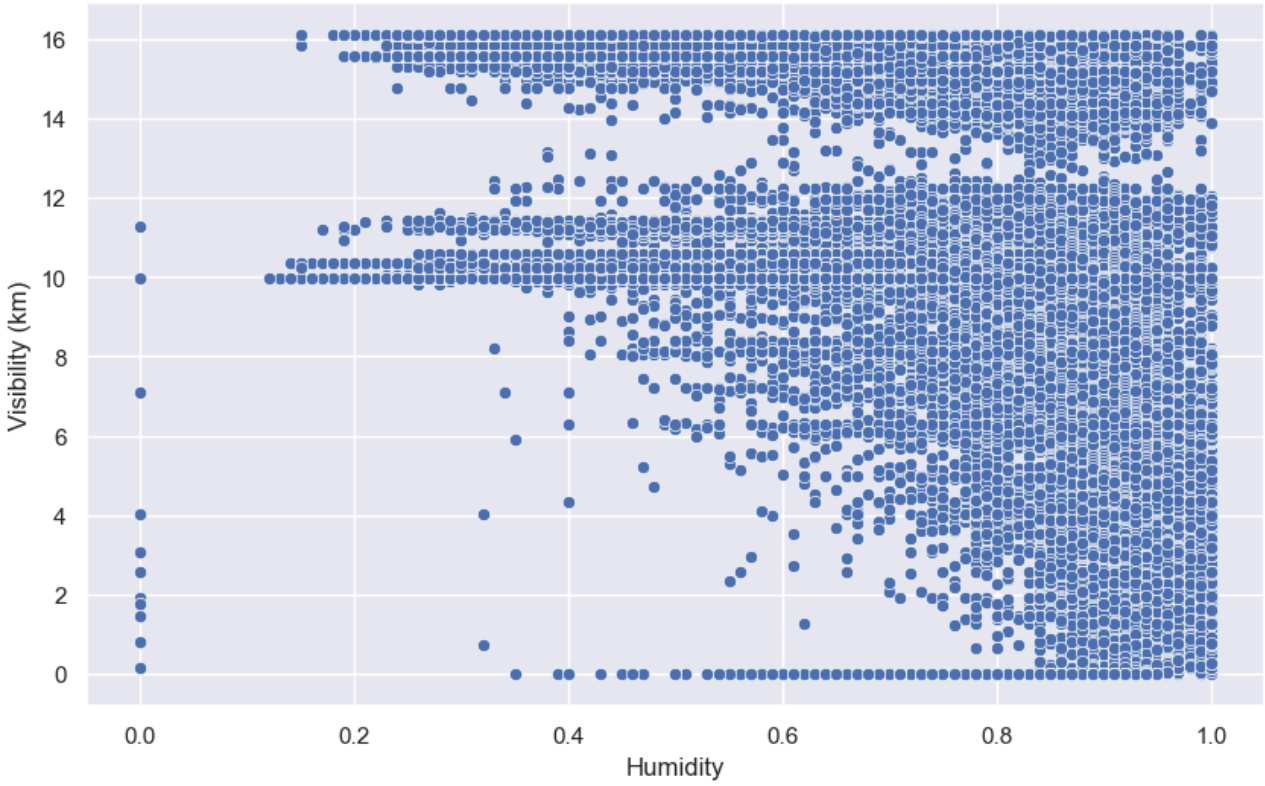
	Temperature (C)	month
Temperature (C)	1.00000	0.14848
month	0.14848	1.00000

This correlation analysis shows a very weak positive relationship between temperature and month. The correlation value of 0.15 indicates that temperature tends to increase slightly as the months progress, reflecting seasonal changes such as warmer temperatures during summer months. However, the relationship is weak, meaning temperature is not completely dependent on the month alone.

Humidity vs Visibility

In [71…
```
sns.scatterplot(x='Humidity',y='Visibility (km)',data=df)
```

Out[71]:
```
<Axes: xlabel='Humidity', ylabel='Visibility (km)'>
```



In [72…
```
df[['Humidity','Visibility (km)']].corr()
```

Out[72]:

	Humidity	Visibility (km)
Humidity	1.000000	-0.369282
Visibility (km)	-0.369282	1.000000

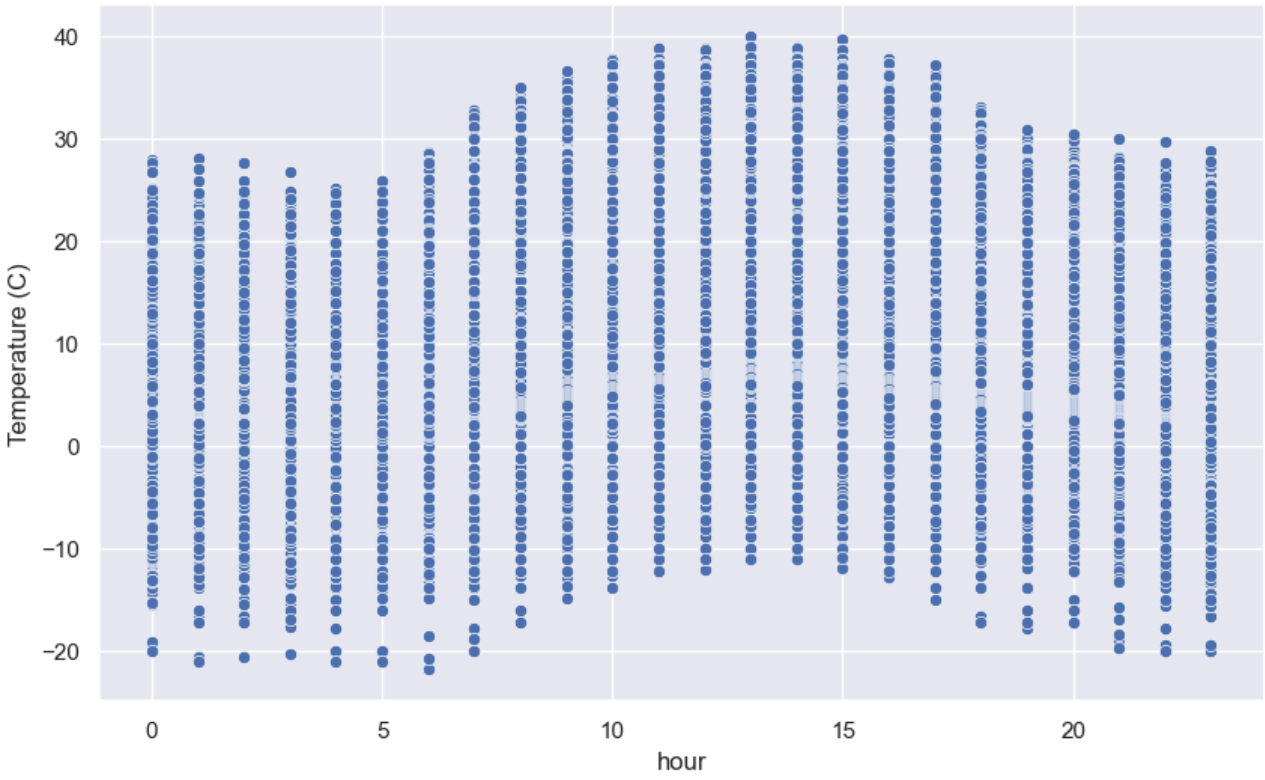
This correlation analysis shows a moderate negative relationship between humidity and visibility. The correlation value of -0.37 indicates that when humidity increases, visibility tends to decrease, and when humidity decreases, visibility improves. This happens because higher moisture in the air often leads to fog, haze, or cloudiness, which reduces how far we can see clearly.

Overall, the dataset shows that more humid conditions are generally associated with lower visibility.

Hour vs Temperature (c)

In [73…
```
sns.scatterplot(x='hour',y='Temperature (C)',data=df)
```

Out[73]:
```
<Axes: xlabel='hour', ylabel='Temperature (C)'>
```



In [74…
```
df[['Temperature (C)','hour']].corr()
```

Out[74]:

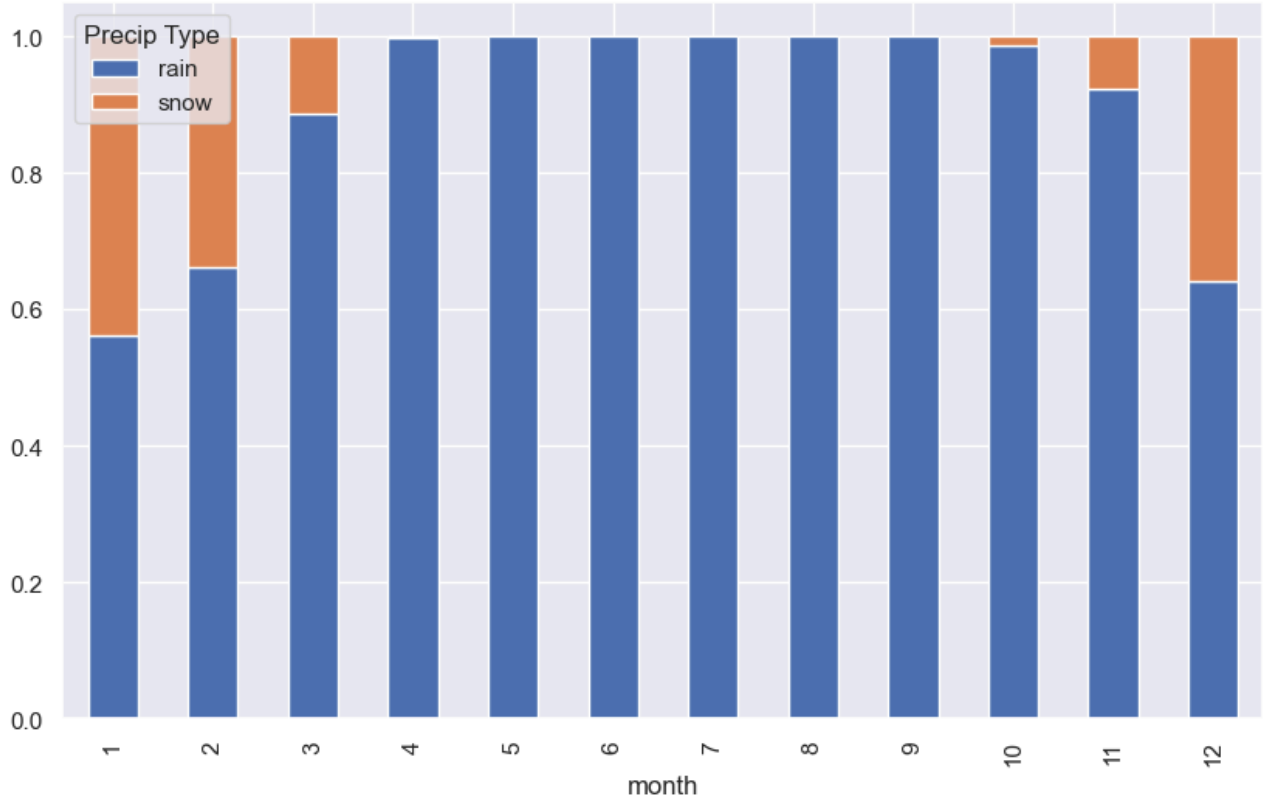
	Temperature (C)	hour
Temperature (C)	1.000000	0.099538
hour	0.099538	1.000000

This correlation analysis shows a very weak positive relationship between temperature and hour. The correlation value of 0.10 indicates that temperature increases slightly at certain hours of the day, suggesting a mild diurnal (day–night) pattern where temperatures may be higher during daytime hours. However, the relationship is weak, meaning hour alone does not strongly determine temperature.

Overall, the dataset shows only a small daily time-based effect on temperature.

month vs Precip Type

In [75…
```
pd.crosstab(df['month'], df['Precip Type'], normalize='index').plot(kind=
plt.show()
```



In [76…
```
pd.crosstab(df['month'], df['Precip Type'], normalize='index') * 100
```

Out[76]:

Precip Type	rain	snow
month		
1	56.256109	43.743891
2	66.224544	33.775456
3	88.636364	11.363636
4	99.722222	0.277778
5	100.000000	0.000000
6	100.000000	0.000000
7	100.000000	0.000000
8	100.000000	0.000000
9	100.000000	0.000000
10	98.607038	1.392962
11	92.222222	7.777778
12	64.210655	35.789345

This bivariate analysis between month and precipitation type (rain/snow) shows a clear seasonal pattern in the dataset. During the summer months (May to September), precipitation is almost entirely in the form of rain (near 100%), with snow being almost absent. In contrast, the winter months (December, January, and February) show a much higher proportion of snow, especially in January and December, where snow makes up a significant share of precipitation. Transitional months like March, April, October, and November show mixed conditions, where both rain and snow are present but rain remains dominant. Overall, the results clearly indicate a strong seasonal cycle, where warmer months are associated with rain and colder months are associated with snow.

Final Summary of Bivariate Analysis

The bivariate analysis of the weather dataset shows clear and meaningful relationships between different weather variables. Temperature is the main driver of weather patterns and shows strong and logical connections with several features. Dry conditions are associated with the highest temperatures, while foggy and overcast conditions are linked with the lowest temperatures. Rain and snow are strongly dependent on temperature, where rain occurs in

warmer conditions and snow only appears in cold conditions.

Humidity shows a clear inverse relationship with both temperature and visibility, meaning higher humidity is linked with cooler temperatures and lower visibility. Visibility improves in warmer and less humid conditions. Wind speed and wind direction show almost no strong relationship with temperature, indicating they behave independently. Atmospheric pressure has a weak negative relationship with temperature, where higher temperatures slightly correspond to lower pressure.

The analysis of time-based features shows that temperature has a weak seasonal and daily pattern. Month has a small but visible effect on temperature, while hour shows a very weak diurnal effect. Precipitation type also shows a strong seasonal pattern, with rain dominating in warmer months and snow appearing mainly in winter months.

Overall, the dataset clearly shows that **temperature and humidity are the most influential factors** in determining weather conditions, while other variables like wind and time have weaker effects. The weather patterns are consistent and follow expected real-world seasonal and atmospheric behavior.

MULTIVARIATE ANALYSIS

Multivariate analysis is performed to understand how multiple atmospheric and time-based variables interact together to influence weather behavior. It helps identify feature dependencies, seasonal patterns, and combined environmental effects within the dataset.

Full Correlation Matrix

In [77…
```
df.corr(numeric_only=True)
```

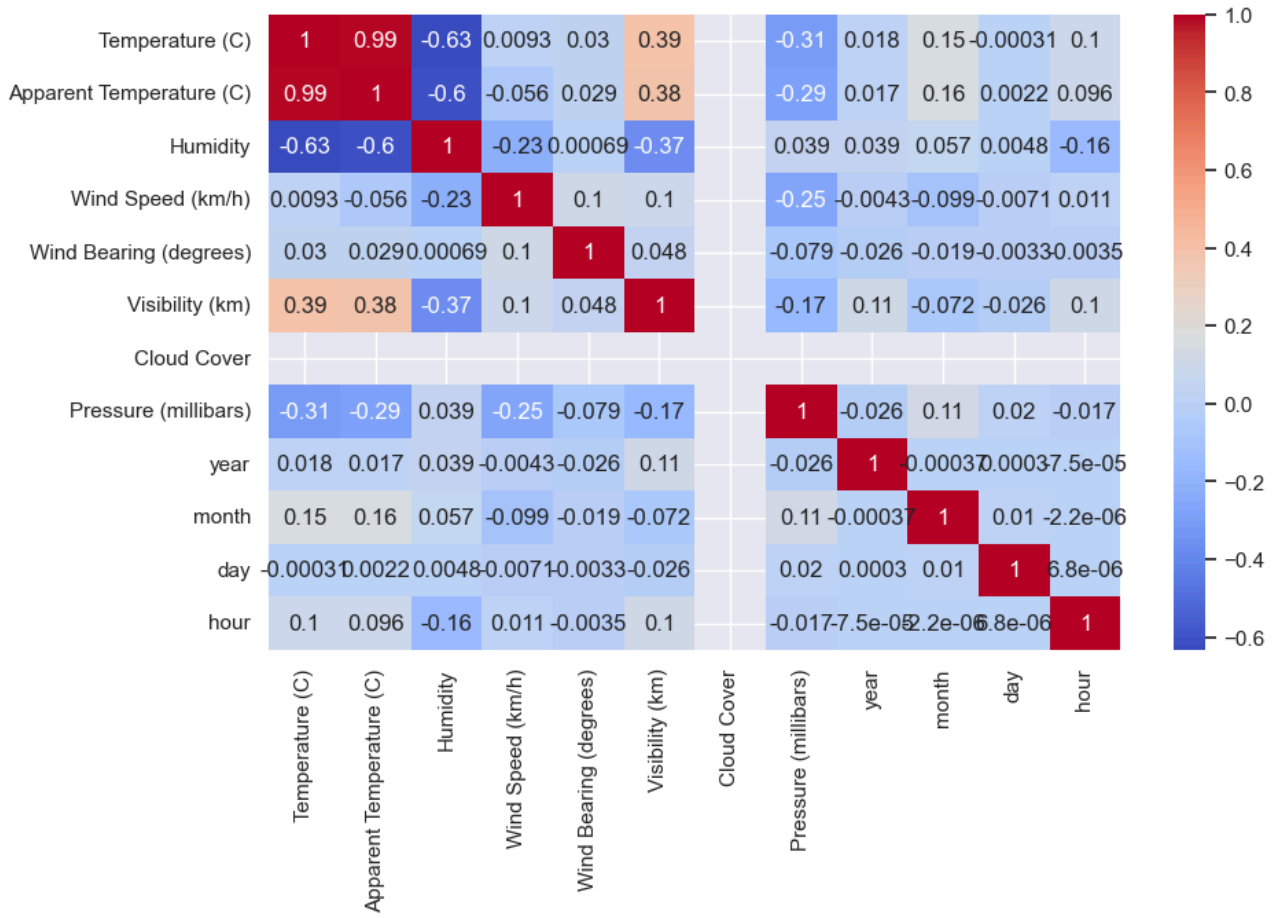
Out[77]:

	Temperature (C)	Apparent Temperature (C)	Humidity	Wind Speed (km/h)	Wind Bearing (degrees)	Visibility (km)	Cloud Cover	(
Temperature (C)	1.000000	0.992625	-0.632331	0.009312	0.029888	0.392939	NaN	
Apparent Temperature (C)	0.992625	1.000000	-0.602641	-0.056324	0.028926	0.381804	NaN	
Humidity	-0.632331	-0.602641	1.000000	-0.225080	0.000689	-0.369282	NaN	
Wind Speed (km/h)	0.009312	-0.056324	-0.225080	1.000000	0.103939	0.100796	NaN	
Wind Bearing (degrees)	0.029888	0.028926	0.000689	0.103939	1.000000	0.047582	NaN	
Visibility (km)	0.392939	0.381804	-0.369282	0.100796	0.047582	1.000000	NaN	
Cloud Cover	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
Pressure (millibars)	-0.310469	-0.288406	0.039253	-0.253791	-0.078685	-0.166331	NaN	
year	0.017565	0.017274	0.039259	-0.004329	-0.025656	0.107317	NaN	
month	0.148480	0.157892	0.056543	-0.098810	-0.019259	-0.071959	NaN	
day	-0.000313	0.002230	0.004831	-0.007075	-0.003319	-0.026343	NaN	
hour	0.099538	0.096035	-0.159931	0.011149	-0.003484	0.103546	NaN	

Correlation Heatmap

In [78…

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```



Correlation Heatmap Summary

The correlation heatmap provides a visual overview of relationships between all numerical weather variables. Dark red colors represent strong positive correlations, while blue colors represent negative correlations.

The strongest positive relationship is observed between **Temperature** and **Apparent Temperature (~0.99)**, showing that both variables increase and decrease together almost identically. Temperature also has a strong negative correlation with **Humidity (~-0.63)**, indicating that warmer conditions are generally associated with lower humidity levels.

Visibility has a moderate positive relationship with temperature and a moderate negative relationship with humidity, meaning clearer visibility is more common in warm and less humid conditions. **Pressure** shows a weak negative relationship with temperature and wind speed, suggesting slightly lower pressure during warmer or windier weather.

Most wind-related variables, such as **Wind Speed** and **Wind Bearing**, show very weak correlations with other features, meaning they do not strongly influence overall weather patterns in the dataset. Time-related features like **month** and **hour** show only weak seasonal or daily effects on temperature. The **Cloud Cover** feature contains missing correlations (NaN values), indicating insufficient or unusable data.

The heatmap shows that the dataset is mainly driven by relationships between **temperature, humidity, apparent temperature, and visibility**, while most other variables have weak or minimal interactions. Overall, the weather patterns follow logical real-world atmospheric behavior.

Temperature Correlation

In [79…
```
df.corr(numeric_only=True)['Temperature (C)'].sort_values(ascending=False
```

Out[79]:
```
Temperature (C)             1.000000
Apparent Temperature (C)    0.992625
Visibility (km)             0.392939
month                       0.148480
hour                        0.099538
Wind Bearing (degrees)      0.029888
year                        0.017565
Wind Speed (km/h)           0.009312
day                        -0.000313
Pressure (millibars)       -0.310469
Humidity                   -0.632331
Cloud Cover                      NaN
Name: Temperature (C), dtype: float64
```

Temperature Correlation Summary

This analysis shows how strongly different variables are related to **Temperature** in the dataset.

Apparent Temperature (~0.99) has the strongest positive correlation with temperature, meaning both variables behave almost identically.

Visibility (~0.39) shows a moderate positive relationship, indicating that warmer conditions are generally associated with clearer visibility.

Month (~0.15) and **Hour (~0.10)** have weak positive relationships with temperature, reflecting small seasonal and daily weather patterns.

Wind Speed, **Wind Bearing**, **Year**, and **Day** show almost no meaningful relationship with temperature.

On the negative side:

Humidity (~-0.63) has the strongest negative correlation, meaning humidity generally decreases as temperature increases.

Pressure (~-0.31) also shows a moderate negative relationship, suggesting slightly lower atmospheric pressure during warmer conditions.

The **Cloud Cover** feature contains missing values (NaN), indicating insufficient data for correlation analysis.

Overall Conclusion

Overall, temperature is most strongly connected with **apparent temperature, humidity, visibility, and pressure**, while most other variables have weak or negligible relationships with it.

Hypothesis 2: Apparent Temperature vs. Actual Temperature

H_2: High wind speeds increase the difference between Actual Temperature and Apparent Temperature

This hypothesis was tested during the Multivariate Analysis phase because it involves the combined relationship between multiple variables:

Temperature

Apparent Temperature

Wind Speed

Multivariate analysis helps in understanding how multiple features together influence weather conditions.

Feature Engineering

A new feature named temp_diff was created to measure the difference

between Actual Temperature and Apparent Temperature.

In [80…
```
df['temp_diff'] = df['Apparent Temperature (C)'] - df['Temperature (C)']
```

Reason:

This feature helps measure how different the “feels-like” temperature is from the actual recorded temperature.

Relationship Analysis

The relationship between Wind Speed and the temperature difference was analyzed using correlation analysis and visualizations.

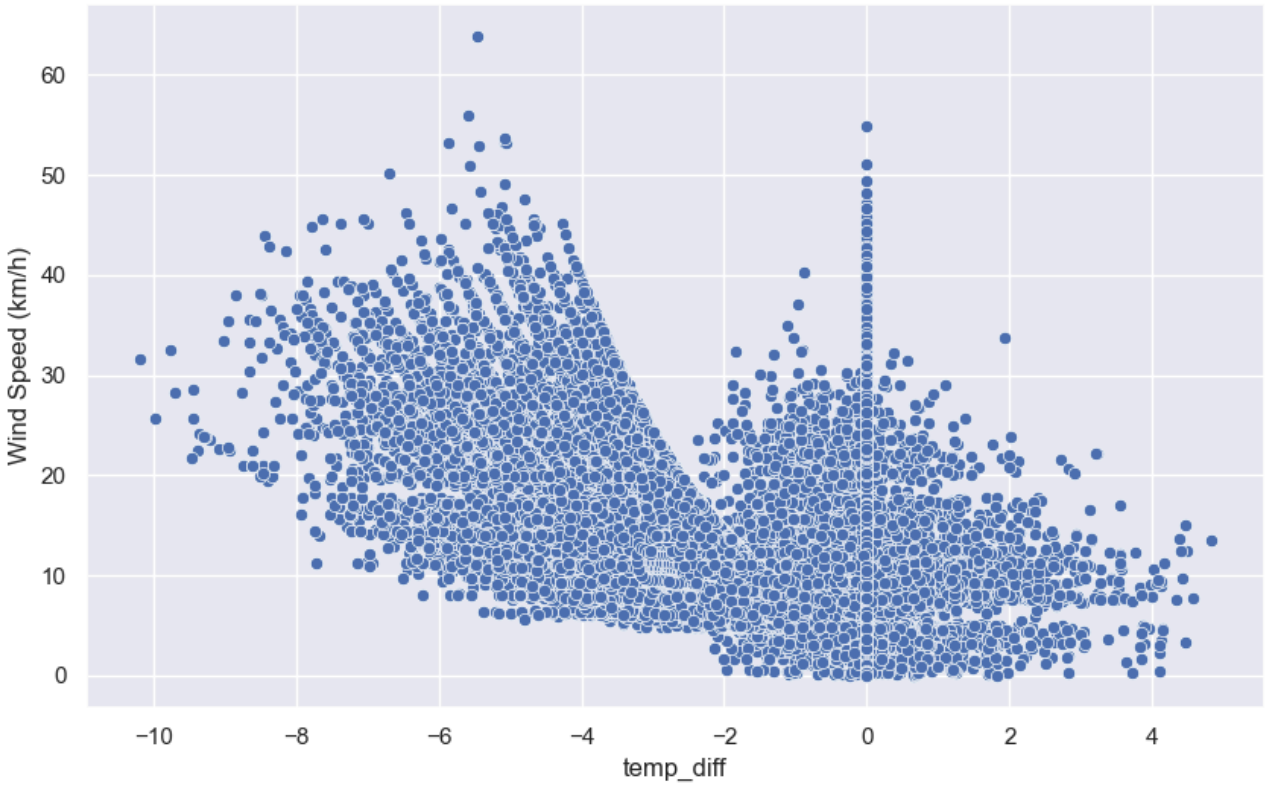
In [81…
```
df[['Wind Speed (km/h)', 'temp_diff']].corr()
```

Out[81]:

	Wind Speed (km/h)	temp_diff
Wind Speed (km/h)	1.00000	-0.41183
temp_diff	-0.41183	1.00000

In [82…
```
sns.scatterplot(x='temp_diff',y='Wind Speed (km/h)', data=df)
```

Out[82]:
```
<Axes: xlabel='temp_diff', ylabel='Wind Speed (km/h)'>
```



Observations

The analysis revealed the following patterns:

At low wind speeds, Actual Temperature and Apparent Temperature remained nearly similar. At higher wind speeds, Apparent Temperature became lower than the Actual Temperature.

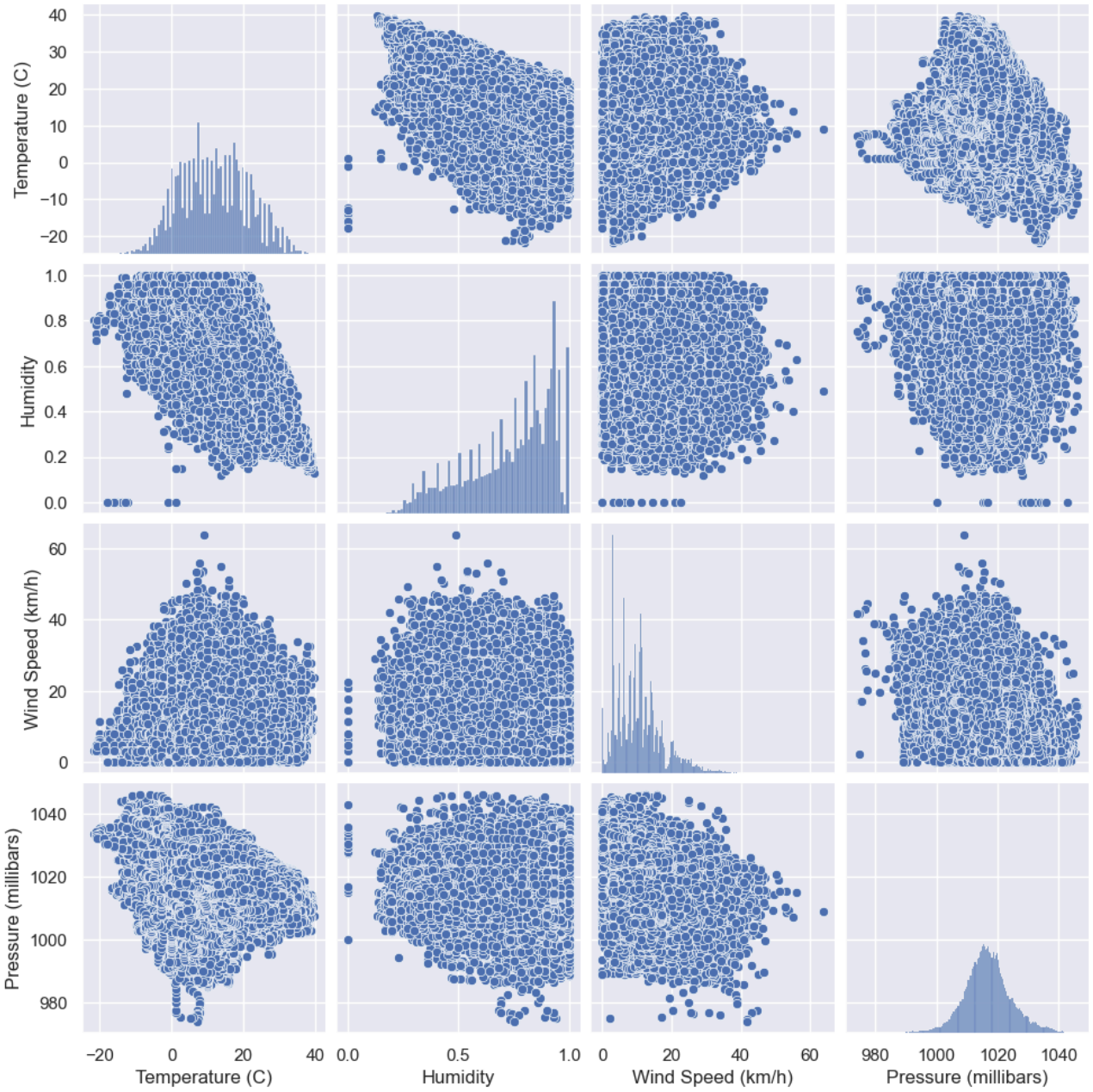
This indicates that stronger winds make the weather feel colder than the measured air temperature.

Multivariate analysis was performed using Wind Speed, Actual Temperature, and Apparent Temperature. The analysis showed that as wind speed increased, the difference between actual temperature and apparent temperature also increased. This indicates the wind chill effect, where strong winds make the weather feel colder than the measured temperature.

Pairplot

In [83…

```
sns.pairplot(df[['Temperature (C)', 'Humidity', 'Wind Speed (km/h)', 'Pre
plt.show()
```



Grouped Multivariate Analysis

In [84…

```
df.groupby('month')[['Temperature (C)', 'Humidity', 'Wind Speed (km/h)']]
```

Out[84]:

month	Temperature (C)	Humidity	Wind Speed (km/h)
1	0.815678	0.850698	11.515865
2	2.166880	0.813407	12.180815
3	6.914976	0.702849	13.415825
4	12.769200	0.641051	11.876139
5	16.874455	0.691650	10.953186
6	20.728510	0.686418	9.632453
7	22.965462	0.639593	9.637124
8	22.334460	0.635044	8.958674
9	17.503458	0.689163	9.610204
10	11.334119	0.774443	10.008495
11	6.587549	0.827917	10.947953
12	1.625086	0.870398	11.094578

Monthly Weather Pattern Summary

This analysis shows how **temperature, humidity, and wind speed change across different months**.

Temperature follows a clear seasonal pattern throughout the year. The coldest months are **January and December**, with average temperatures around **1–2°C**, while the warmest months are **July and August**, where temperatures rise to around **22–23°C**. This indicates a strong transition from winter to summer seasons.

Humidity behaves in the opposite way compared to temperature. Winter months such as **December and January** have the highest humidity levels (0.85–0.87), while summer months like **July and August** show lower humidity (~0.63–0.64). This suggests that colder weather is generally more humid, while warmer weather is relatively drier.

Wind speed remains fairly stable throughout the year, mostly ranging between **9–13 km/h**. Slightly higher wind speeds are observed during late winter and early spring months, while summer months experience relatively calmer wind conditions.

The dataset shows a clear seasonal climate cycle:

Winter months → colder and more humid

Summer months → warmer and less humid

Wind speed remains mostly moderate throughout the year

Overall, temperature and humidity display strong seasonal variation, while wind speed changes only slightly across months.

The monthly weather pattern analysis further supported the **seasonality hypothesis 3**. Humidity levels showed clear variation across different months and seasons. Winter months exhibited higher humidity levels, while summer months showed comparatively lower humidity. This confirms that atmospheric moisture is strongly influenced by seasonal climatic changes.

Pivot Table

In [85…
```
df.pivot_table(values='Temperature (C)',
               index='month',
               columns='Precip Type',
               aggfunc='mean')
```

Out[85]:

Precip Type	rain	snow
month		
1	4.223430	-3.566806
2	5.208673	-3.797256
3	8.065434	-2.058590
4	12.808118	-1.202273
5	16.874455	NaN
6	20.728510	NaN
7	22.965462	NaN
8	22.334460	NaN
9	17.503458	NaN
10	11.513499	-1.364133
11	7.298124	-1.837834
12	4.332410	-3.232199

Monthly Temperature Pattern by Precipitation Type

This analysis shows how temperature changes across months for both **rain** and **snow** conditions.

Rain occurs throughout the year and follows a clear seasonal temperature pattern. The average temperature during rainy conditions is lowest in **winter months** such as January and December (4–5°C) and gradually increases toward summer, reaching the highest values in **July and August (22–23°C)**. This indicates that rainfall is common in both cold and warm seasons but becomes warmer during summer months.

Snow is mainly concentrated in colder months. Snow-related temperatures remain below 0°C in almost all months where snowfall occurs, with the coldest conditions observed during **January, February, and December (-3 to -4°C)**. Snow is completely absent during warmer months from **May to September**, which is why those months contain missing values (NaN).

The analysis clearly highlights a strong seasonal relationship between precipitation type and temperature:

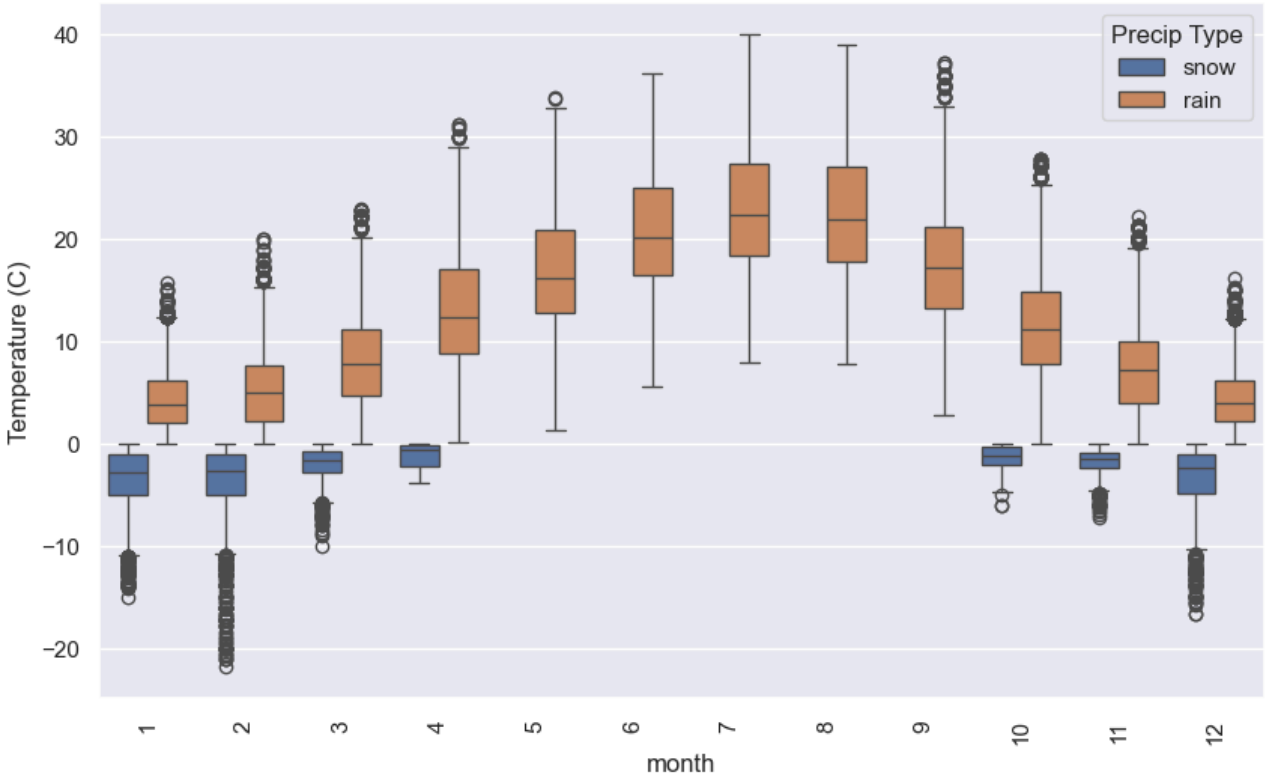
Rain occurs across all seasons but becomes warmer in summer months Snow occurs only in very cold conditions and disappears during warm months

Overall, temperature plays a major role in determining whether precipitation falls as rain or snow throughout the year.

Multi-feature Visualization

In [86…

```
sns.boxplot(x=df['month'], y=df['Temperature (C)'], hue=df['Precip Type'])
plt.xticks(rotation=90)
plt.show()
```



Multi-Feature Visualization Summary

This boxplot visualizes the relationship between **Month, Temperature, and Precipitation Type (Rain/Snow)** simultaneously.

The plot clearly shows a strong seasonal weather pattern throughout the year. Rain-related temperatures gradually increase from winter to summer, reaching their highest levels during **June, July, and August**, and then decrease again toward winter months. Rain occurs across almost all months, but temperatures during rainy conditions are much warmer in summer.

Snow-related temperatures remain consistently below 0°C and are mostly concentrated in colder months such as **January, February, November, and December**. Snow is nearly absent during summer months, confirming that snowfall only occurs under cold temperature conditions.

The boxplot also shows temperature variability:

- Summer months have a wider spread of warm temperatures
- Winter months contain colder temperatures and more extreme low-temperature outliers

- Rain has much larger temperature variation compared to snow

The visualization clearly demonstrates that:

Rain is associated with warmer temperatures and occurs throughout most of the year Snow is limited to cold winter months with sub-zero temperatures Temperature strongly influences both seasonal patterns and precipitation type

Overall, the dataset follows realistic seasonal climate behavior with clear separation between rain and snow conditions.

Final Summary — Multivariate Analysis

The multivariate analysis examines how multiple weather variables interact with one another across different seasons and weather conditions. The findings highlight **Temperature as one of the key variables associated with changes in overall weather patterns** within the dataset.

A very strong positive relationship is observed between **Temperature and Apparent Temperature**, suggesting that both variables follow similar patterns. In contrast, **Humidity generally shows a negative relationship with Temperature**, indicating that higher temperatures in this dataset tend to be associated with lower humidity levels. Visibility also varies with temperature and humidity, with foggy or highly humid conditions often associated with reduced visibility.

The analysis reveals noticeable seasonal variations in weather conditions. **Summer is generally associated with higher temperatures**, while winter is characterized by colder conditions and a greater occurrence of snow. Rainfall occurs across multiple months and under a broader range of temperatures, whereas snowfall is primarily associated with colder conditions. These patterns demonstrate how precipitation types vary with temperature and season.

Wind-related variables, including **Wind Speed and Wind Bearing**, show comparatively weaker relationships with several other weather variables. Atmospheric pressure also exhibits some variation in relation to temperature, although the strength and direction of this relationship should be interpreted

based on the observed data.

The multivariate visualizations further illustrate the combined relationships among temperature, humidity, visibility, and precipitation type. In particular, **temperature helps distinguish the conditions under which rain and snow occur**, while seasonal comparisons provide additional context for understanding changes in weather patterns throughout the year.

Overall, the multivariate analysis provides a broader understanding of the relationships between atmospheric variables and seasonal weather conditions. **Temperature, Apparent Temperature, Humidity, Visibility, and Precipitation Type emerge as important variables for further investigation.** These findings provide a useful foundation for interpreting historical weather patterns and conducting more detailed statistical analysis.

Hypothesis Testing Summary

The hypothesis testing phase aims to examine the relationships between selected weather variables and determine whether the observed patterns are supported by statistical evidence. The analysis focuses on temperature, humidity, apparent temperature, wind speed, and seasonal variations to better understand their interactions within the dataset.

Hypotheses and Findings

H_1: Relationship Between Temperature and Humidity

The analysis investigates the relationship between Temperature and Relative Humidity. A negative association may indicate that higher temperatures tend to occur alongside lower humidity levels in the observed data. The strength and statistical significance of this relationship should be evaluated using correlation analysis and an appropriate statistical test.

H_2: Relationship Between Apparent Temperature and Wind Speed

This hypothesis examines whether wind speed is associated with the difference between Actual Temperature and Apparent Temperature. Higher wind speeds may contribute to a greater perceived temperature difference, particularly under colder conditions. Further statistical testing is required to determine whether this relationship is significant in the dataset.

H_3: Seasonal Variation in Humidity

This hypothesis examines whether humidity levels differ across seasons and months. Comparing humidity distributions across different time periods can help identify seasonal patterns and determine whether the observed differences are statistically significant.

Overall Interpretation

The hypothesis testing process provides a structured way to evaluate relationships among weather variables and distinguish observed patterns from statistically supported findings. Temperature, humidity, apparent temperature, wind speed, and seasonal changes are important areas of investigation in understanding historical weather behavior.

The final conclusions should be based on the statistical test results, including the selected significance level, p-values, and relevant test statistics. This ensures that each hypothesis is classified appropriately as supported or not supported by the available evidence.

Business Impact and Practical Applications

Weather Forecasting: Understanding relationships between weather variables can support the development and evaluation of forecasting models.

Disaster Preparedness: Historical weather patterns can help identify periods associated with extreme cold, heavy snowfall, or other adverse weather conditions, supporting better preparedness and planning.

Transportation and Logistics: Insights into seasonal weather variations can help transportation and logistics teams anticipate potential disruptions and plan operations more effectively.

Agricultural Planning: Weather trend analysis can provide useful information for agricultural planning, including decisions related to crop cycles, irrigation, and seasonal activities.

Infrastructure Management: Understanding temperature, precipitation, and visibility patterns can support planning for drainage systems, road maintenance, and weather-sensitive infrastructure.

Energy Demand Planning: Historical temperature trends can help energy providers understand seasonal changes in electricity demand associated with heating and cooling requirements.

Final Summary – Exploratory Data Analysis of Weather Data

The exploratory data analysis (EDA) of the historical weather dataset was conducted through three major stages: **Univariate Analysis, Bivariate Analysis, and Multivariate Analysis**. The objective was to understand the dataset's structure, assess weather variable distributions, identify seasonal patterns, and examine relationships among different meteorological factors.

1. Key Findings from Univariate Analysis

Univariate analysis helped examine the distribution and characteristics of individual weather variables. It provided an overview of temperature,

apparent temperature, humidity, wind speed, visibility, atmospheric pressure, and precipitation type. The analysis also highlighted variations in weather conditions and the importance of considering time and season when interpreting historical observations.

2. Key Findings from Bivariate Analysis

Bivariate analysis explored relationships between pairs of weather variables. Temperature and Apparent Temperature showed a strong positive relationship, while Temperature and Humidity exhibited a negative association in the observed data. The analysis also helped investigate how visibility, wind conditions, and precipitation types vary with temperature and other atmospheric factors.

3. Key Findings from Multivariate Analysis

Multivariate analysis provided a broader view of how several weather variables interact simultaneously. The results highlighted the importance of considering temperature, humidity, visibility, precipitation type, and seasonal variations together rather than examining each variable in isolation. These combined relationships offered a more comprehensive understanding of the weather patterns represented in the dataset.

4. Overall Conclusions

The exploratory analysis provided the following key takeaways:

1. **Weather patterns vary over time:** Seasonal and temporal comparisons help reveal changes in atmospheric conditions.
2. **Temperature is an important variable:** It shows a strong relationship with Apparent Temperature and is useful for investigating other weather characteristics.
3. **Precipitation varies with weather conditions:** Rain and snow occur under different temperature ranges and seasonal conditions.
4. **Weather variables are interconnected:** Examining multiple variables together provides a broader understanding than analysing individual variables alone.
5. **Data visualization supports interpretation:** Distribution plots, relationship charts, and multivariate visualizations help communicate patterns and potential anomalies.

6. **Statistical analysis adds evidence:** Appropriate statistical tests can help determine whether observed relationships and group differences are statistically significant.

5. Business Relevance

The findings from this project may support practical decision-making in areas such as weather forecasting, transportation and logistics planning, agricultural operations, infrastructure management, disaster preparedness, and energy demand planning. However, these are potential applications of the insights rather than outcomes directly measured by this exploratory analysis.

6. Limitations and Future Scope

Although the analysis provides useful insights into historical weather patterns, exploratory data analysis alone cannot establish causation or guarantee accurate future predictions. The findings are also limited to the variables, time period, geographic coverage, and data quality of the available dataset.

Future work could extend this project through the following activities:

- **Time-Series Analysis:** Investigate long-term trends, recurring seasonal patterns, and changes over time.
- **Advanced Statistical Testing:** Apply suitable hypothesis tests and report p-values and effect sizes to evaluate specific relationships.
- **Feature Engineering:** Create additional features such as month, season, hour, temperature difference, and other relevant time-based indicators.
- **Predictive Modelling:** Evaluate suitable machine learning or time-series models for forecasting selected weather variables.
- **Model Evaluation:** Compare predictions against actual observations using appropriate evaluation metrics.
- **Interactive Dashboard:** Develop a Power BI dashboard to explore weather trends, seasonal comparisons, and variable relationships interactively.

7. Final Conclusion

This project demonstrates how Python-based exploratory data analysis can transform raw historical weather observations into meaningful analytical insights. Through data exploration, visualization, relationship analysis, and

statistical investigation, the project builds a foundation for understanding weather variability and identifying patterns among meteorological variables.

Overall, the project establishes a structured analytical foundation for further statistical investigation, time-series analysis, and potential predictive modelling. It also demonstrates the practical value of data cleaning, exploratory analysis, and visualization in extracting insights from real-world datasets.